



**國立臺北科技大學**

**資訊工程系碩士班**

**碩士學位論文**

**物流公司支援線規劃之多階段最佳化方法**

**Multi-Stage Optimization for Supporting Lines in  
Logistics Companies**

**研究生：楊宏傑**

**指導教授：郭忠義 博士**

**中華民國一百一十五年七月**



# 摘要

關鍵字：車輛路徑問題、基因演算法、蟻群演算法、路徑最佳化

現行物流配送流程中，路線編排仍仰賴人工經驗，不僅耗費大量時間成本，亦難以確保路線規劃之品質與效率。本研究聚焦於時間窗車輛路徑問題(Vehicle Routing Problem with Time Windows, VRPTW)，以物流公司提供之初步規劃路線為基礎，提出一套 Multi-stage Operationally Constrained VRP Framework，以分階段方式處理實務配送情境中之超載調整、店鋪重新分配與支援線規劃問題。

本研究透過歸納物流公司之實務規劃經驗，定義主線與支援線之編排規則，其後根據各階段問題特性，本研究分別採用合作式協同演化基因演算法(Cooperative Co-evolutionary Genetic Algorithm, CCGA)、混合式蟻群演算法(Hybrid Ant Colony Optimization, HACO)以及多蟻群蟻群最佳化(Multiple Ant Colony System, MACS)作為各階段之求解策略，以提升整體求解品質與效率。

為驗證方法之成效，本研究以物流公司歷史人工編排路徑作為實驗比較基準，透過對比系統最佳化路線與人工路徑規劃路線之差異，以此判斷能否提升配送效益，並符合物流配送環境之限制。實驗結果顯示，透過比較 26 組人工編排之配送路線資料，本研究提出之方法相較於人工規劃方式，能有效提升整體營運效益。結果顯示，平均可減少約 7.00%的路線需求、降低約 3.07%的總行駛距離，並使路線平均裝載率提 6.70%，準點率亦提升平均 7.16%。上述結果顯示，本研究方法在降低營運成本之同時，亦能提升配送路線之規劃品質。

# ABSTRACT

Keywords: Vehicle Routing Problem, Genetic Algorithm, Ant Colony Optimization, Route Optimization

Route planning in logistics operations remains highly dependent on human experience, leading to significant operational costs and challenges in ensuring routing quality and efficiency. This study focuses on the Vehicle Routing Problem with Time Windows (VRPTW). Based on initial routes provided by a logistics company, a Multi-stage Operationally Constrained VRP Framework is proposed to sequentially address store extraction from overloaded routes, store assignment, and support route planning in practical distribution environments.

The proposed framework incorporates practical route-planning knowledge by defining operational rules for main and support routes. To enhance solution quality, the Cooperative Co-evolutionary Genetic Algorithm (CCGA), Hybrid Ant Colony Optimization (HACO), and Multiple Ant Colony System (MACS) are employed as stage-specific optimization strategies.

Experimental results based on 26 delivery instances demonstrate that the proposed framework achieves superior performance compared with manual planning. On average, the number of required routes is reduced by 7.00%, total travel distance is decreased by 3.07%, route utilization is increased by 6.70%, and on-time delivery performance is improved by 7.16%. These results indicate that the proposed framework effectively enhances routing quality while reducing operational costs.

## 致謝

首先，誠摯感謝我的指導教授郭忠義博士。感謝老師在研究所期間悉心指導，不僅在研究方法、專業知識及論文撰寫上給予耐心的教導，更時常以自身經驗教導我們做人處事的態度與責任感。

感謝口試委員薛念林教授、馬尚彬教授及范姜永益教授，於百忙之中撥冗審閱本論文並參與口試，提出許多寶貴且具建設性的建議，使本論文得以更加完整，也讓我對研究內容有更深入思考。在此向三位教授致上最誠摯的謝意。

感謝實驗室所有學長姐及學弟妹們，在研究所求學期間給予我許多協助與陪伴。特別感謝同屆同學楊其康、賴星洲、陳泰吉、陳勝誠、陳至弘、王映儒、陳宜滇及徐家玟。無論是在研究上的討論與交流、課業上的互相學習，或是生活中的彼此關心與陪伴，都讓我受益良多。感謝學長姐不吝分享研究經驗，使我在研究過程中少走許多彎路。感謝同學們在面對壓力與挑戰時彼此扶持。

感謝我的家人與朋友。每當我因研究受挫而感到徬徨時，總是因為有你們的陪伴與鼓勵，讓我能夠重新振作，勇敢面對每一次挑戰，並堅持完成自己的目標。

研究所生活即將畫下句點，而這段旅程所累積的知識、經驗與回憶，將成為人生中珍貴的養分。謹以此文，向所有曾經給予我指導、幫助、支持與鼓勵的師長、同學、家人及朋友，致上最誠摯的感謝，並祝福大家未來都能朝著自己的理想邁進。

# 目錄

|                                                                          |      |
|--------------------------------------------------------------------------|------|
| 摘要.....                                                                  | i    |
| ABSTRACT.....                                                            | ii   |
| 致謝.....                                                                  | iii  |
| 目錄.....                                                                  | iv   |
| 表目錄.....                                                                 | vii  |
| 圖目錄.....                                                                 | viii |
| 第一章 緒論.....                                                              | 1    |
| 1.1 研究動機與目的.....                                                         | 1    |
| 1.2 研究貢獻.....                                                            | 2    |
| 1.3 章節規劃.....                                                            | 3    |
| 第二章 背景知識與文獻探討.....                                                       | 4    |
| 2.1 Vehicle Routing Problem.....                                         | 4    |
| 2.1.1 Capacitated Vehicle Routing Problem.....                           | 4    |
| 2.1.2 Vehicle Routing Problem with Time Windows.....                     | 5    |
| 2.1.3 Vehicle Routing Problem with Private Fleet and Common Carrier..... | 6    |
| 2.2 Genetic Algorithm.....                                               | 8    |
| 2.2.2 Cooperative Co-evolutionary Genetic Algorithm.....                 | 9    |
| 2.3 Ant Colony Optimization.....                                         | 10   |
| 2.3.1 Hybrid Ant Colony Optimization.....                                | 11   |
| 2.4 現有方法之限制與比較探討.....                                                    | 12   |
| 2.4.1 現有 VRP 模型與實務需求之差異.....                                             | 12   |
| 2.4.2 單一演算法局限性與分階段設計.....                                                | 14   |
| 第三章 研究方法.....                                                            | 16   |
| 3.1 問題實例描述.....                                                          | 16   |
| 3.1.1 符號定義.....                                                          | 18   |
| 3.2 系統架構.....                                                            | 24   |
| 3.2.1 原始配送路線資料 (Original Route Data).....                                | 25   |

|                                                         |    |
|---------------------------------------------------------|----|
| 3.2.2 資料模組 (Data Module).....                           | 25 |
| 3.2.3 路線模組 (Route Module).....                          | 26 |
| 3.2.4 評估模組 (Evaluation Module).....                     | 27 |
| 3.2.5 API 模組 (API Module).....                          | 28 |
| 3.2.6 最佳化配送路線資料 (Optimized Route Data).....             | 28 |
| 3.3 Cooperative Co-evolutionary Genetic Algorithm ..... | 29 |
| 3.3.1 編碼(Encoding).....                                 | 29 |
| 3.3.2 選擇(Selection).....                                | 29 |
| 3.3.3 交配(Crossover).....                                | 29 |
| 3.3.4 突變(Mutation) .....                                | 30 |
| 3.3.5 適應值(Fitness).....                                 | 30 |
| 3.3.6 解的修復或可行性處理.....                                   | 30 |
| 3.4 Hybrid Ant Colony Optimization.....                 | 31 |
| 3.4.1 候選店鋪插入機制.....                                     | 31 |
| 3.4.2 費洛蒙初始化與更新.....                                    | 32 |
| 3.4.3 Variable Neighborhood Descent.....                | 32 |
| 3.4.4 目標函數值.....                                        | 33 |
| 3.5 Multiple Ant Colony System.....                     | 34 |
| 3.5.1 ACS-VEI.....                                      | 34 |
| 3.5.2 ACS-DIST.....                                     | 34 |
| 3.5.3 店鋪節點轉移策略.....                                     | 34 |
| 3.5.4 費洛蒙初始化與更新.....                                    | 35 |
| 3.5.5 Variable Neighborhood Descent.....                | 35 |
| 3.6 Complexity and Problem Decomposition Analysis.....  | 36 |
| 第四章 實驗與討論.....                                          | 38 |
| 4.1 實驗環境.....                                           | 38 |
| 4.2 實驗資料集 .....                                         | 38 |
| 4.3 評估基準 .....                                          | 40 |

|                         |    |
|-------------------------|----|
| 4.4 超參數設定與校準 .....      | 41 |
| 4.4.1 CCGA 之超參數設定 ..... | 41 |
| 4.4.2 HACO 之超參數設定 ..... | 41 |
| 4.4.3 MACS 之超參數設定 ..... | 42 |
| 4.5.1 資料前處理 .....       | 43 |
| 4.5.2 超載路線店鋪抽取 .....    | 45 |
| 4.5.3 候選店鋪分配 .....      | 46 |
| 4.5.4 支援線規劃 .....       | 48 |
| 4.5.5 路線整合與評估 .....     | 51 |
| 4.6 實驗結果比較 .....        | 52 |
| 4.6.1 符號定義 .....        | 52 |
| 4.6.2 實驗結果比較 .....      | 52 |
| 4.6.3 實例研究與分析 .....     | 56 |
| 4.6.4 消融實驗與比較分析 .....   | 61 |
| 4.7 實驗限制與討論 .....       | 71 |
| 第五章 結論與未來研究方向 .....     | 72 |
| 5.1 結論 .....            | 72 |
| 5.2 未來研究方向 .....        | 72 |
| 參考文獻 .....              | 73 |



## 表目錄

|                                             |    |
|---------------------------------------------|----|
| 表 1 演算法對應關係表.....                           | 37 |
| 表 2 實驗設備規格表.....                            | 38 |
| 表 3 CCGA 超參數設定.....                         | 41 |
| 表 4 HACO 超參數設定.....                         | 42 |
| 表 5 蟻群演算法超參數設定.....                         | 42 |
| 表 6 所提出方法與物流公司人工編排比較結果.....                 | 53 |
| 表 7 所提出方法平均改善率.....                         | 54 |
| 表 8 系統與物流公司人工編排平均編排時間.....                  | 54 |
| 表 9 D20260102 整體配送結果比較表.....                | 56 |
| 表 10 D20260102 路線 2M 摘要比較表.....             | 57 |
| 表 11 D20260102 手動編排之路線 2M 店鋪資訊表.....        | 57 |
| 表 12 D20260102 系統編排之路線 2M 店鋪資訊表.....        | 58 |
| 表 13 實驗比較方法說明.....                          | 61 |
| 表 14 Friedman 檢定之方法平均排名結果.....              | 64 |
| 表 15 Friedman 統計檢定結果.....                   | 64 |
| 表 16 Wilcoxon Signed-Rank Test 成對比較結果表..... | 65 |
| 表 18 本研究與基準方法於 C 類之比較.....                  | 66 |
| 表 19 本研究與基準方法於 R 類之比較.....                  | 67 |
| 表 19 本研究與基準方法於 RC 類之比較.....                 | 68 |

# 圖目錄

|                                    |    |
|------------------------------------|----|
| 圖 1 系統架構圖 .....                    | 24 |
| 圖 2 物流公司人工編排路線資料 .....             | 39 |
| 圖 3 原始路線各實例之主線車輛數量 .....           | 39 |
| 圖 4 原始路線各實例之店鋪數量 .....             | 40 |
| 圖 5 原始路線各實例車輛裝載率分佈 .....           | 40 |
| 圖 6 路線資料格式範例 .....                 | 44 |
| 圖 7 店鋪抽取之初始子族群生成偽程式碼 .....         | 45 |
| 圖 8 店鋪抽取之 CCGA 偽程式碼 .....          | 46 |
| 圖 9 HACO 之偽程式碼 .....               | 46 |
| 圖 10 HACO 路線建構之偽程式碼 .....          | 47 |
| 圖 11 可變鄰域下降法之偽程式碼 .....            | 48 |
| 圖 12 MACS 之偽程式碼 .....              | 49 |
| 圖 13 ACS-VEI 之偽程式碼 .....           | 50 |
| 圖 14 ACS-DIST 之偽程式碼 .....          | 50 |
| 圖 15 MACS 路線建構之偽程式碼 .....          | 51 |
| 圖 16 所提出方法各實例之編排時間 .....           | 55 |
| 圖 17 所提出方法各實例於店鋪抽取階段之抽取店鋪數量 .....  | 55 |
| 圖 18 D20260102 手動編排之路線 2M 結果 ..... | 58 |
| 圖 19 D20260102 系統編排之路線 2M 結果 ..... | 59 |
| 圖 20 CCGA 之適應度收斂曲線圖 .....          | 59 |
| 圖 21 HACO 之成本收斂曲線圖 .....           | 60 |
| 圖 22 MACS 之成本收斂曲線圖 .....           | 60 |

|                                |    |
|--------------------------------|----|
| 圖 23 消融實驗於各實例之總車輛數量比較結果.....   | 62 |
| 圖 24 消融實驗於各實例之總行駛距離比較結果.....   | 62 |
| 圖 25 消融實驗於各實例之車輛裝載率比較結果.....   | 63 |
| 圖 26 消融實驗於各實例之店鋪準點率比較結果.....   | 63 |
| 圖 27 本研究與基準方法於各類資料之車輛數量比較..... | 70 |
| 圖 28 本研究與基準方法於各類資料之距離比較.....   | 70 |



# 第一章 緒論

本章旨在說明本研究之整體脈絡，內容包括研究背景、研究動機、研究目標與預期研究成果。介紹研究所處之應用情境與問題，最後說明本論文各章節之規劃。

## 1.1 研究動機與目的

隨著電子商務的迅速發展與消費者需求的多樣性增加，物流公司需在市場中不斷創新，以確保物流配送的穩定性與客戶的滿意度，在此背景下，有效地進行配送規劃已成為物流產業的重要挑戰。車輛路徑問題(Vehicle Routing Problem, VRP)[1]屬 NP-hard 問題，其計算複雜度呈指數成長，使整數線性規劃難以在合理的時間內獲得有效解。因此通常採用元啟發式演算法(Metaheuristic Algorithm)[2]，如基因演算法(Genetic Algorithm, GA)[3]、蟻群演算法(Ant Colony Optimization, ACO)[4]等，以兼顧全域搜尋能力與解的品質。近年來，為進一步提升搜尋效率與解的穩定性，研究者提出混合元啟發式演算法(Hybrid Metaheuristics)[5]，透過結合不同演算法的優勢，以在合理時間內取得更接近全域最佳解的配送路線。

「A 物流公司」為全家便利商店旗下的物流公司，負責將商品配送至全台各地的店鋪。全省分成七個物流中心，每個物流中心均負責所轄區域的配送作業。以林口物流中心(Delivery Center, DC)為例，每日需配送約 800 家店鋪，所需規劃的主線配送路線約 81 條，每條路線涵蓋約 3 至 15 家店鋪。

在實務運作情境中，物流公司多已建立長期穩定之固定主線配送架構，每一條主線皆包含數個已事先規劃完成且服務順序不可隨意變動之店鋪，整體決策邏輯亦具有明確之成本階層結構，即優先於既有主線內部消化配送需求，僅於必要時方啟用支援線進行配送。在此架構下，店鋪每日實際配送需求可能因市場波動而產生變動，進而導致原先可行之主線配置出現超載情形。當主線總配送量超過車輛可承載容量時，系統必須在維持既有服務順序之前提下，選擇部分店鋪進行抽離，並將其重新分配至其他尚具餘裕容

量之主線，若內部資源仍無法吸收超額需求，則需進一步規劃支援線以完成配送任務。

在 VRP 相關研究中，多數文獻著重於時間窗限制、容量限制或車隊組成等單一或特定營運條件之建模與求解。如容量限制車輛路徑問題(Capacitated Vehicle Routing Problem, CVRP)[6]、時間窗車輛路徑問題(Vehicle Routing Problem with Time Windows, VRPTW)[7]及自有車隊與委外車隊問題(Vehicle Routing Problem with Private fleet and Common carrier, VRPPC)[8]等，分別聚焦於容量限制、時間窗約束或車隊選擇之成本權衡。此類研究多假設配送路線可於規劃階段進行全域性調整，較少考量既有固定主線架構下之順序限制與階層式決策邏輯。因此，本研究所面對之問題並非典型可進行全域重排之車輛路徑最佳化問題，而是在固定順序限制與階層式決策邏輯下之部分彈性 VRP，其同時涵蓋子集合選擇與路線再配置及支援線規劃之決策特性，使其在數學模型建構與演算法設計上皆具較高複雜度與顯著實務挑戰。

基於上述背景與挑戰，本研究提出 Multi-stage Operationally Constrained VRP 分階段架構，將主線抽取、分配與支援線規劃視為不同性質的子問題，依其特性配置適切演算法策略，建構兼具理論與實務的配送路線規劃系統，從而提升整體物流配送的效率，降低配送成本並滿足客戶個別化的服務需求，以利物流產業的發展與競爭力提升。

## 1.2 研究貢獻

本研究以某物流公司之配送資料為基礎，針對既有固定主線條件下因配送需求波動所衍生之超載與支援線調度問題，提出一套兼顧實務可行性與最佳化效益的配送路徑規劃方法。相較於傳統車輛路徑問題假設路線可自由重排，本研究納入主線店舖順序不可隨意更動之限制，將主線與支援線調度問題形式化為具時間窗與容量限制且結合自有與委外車隊特性的車輛路徑規劃問題，使問題轉化為多階段組合最佳化決策架構。本研究之具體貢獻如下：

1. 提出固定主線條件下之配送調度問題之形式化模型，將其分解為三種組合最佳化子問題，包括子集合選擇、分配問題與 VRPTW，並針對不同子問題進行搜尋空間複雜度分析，提供理論依據以說明問題難度，為後續演算法設計建立基礎。
2. 設計分階段最佳化架構，針對抽取、分配與支援線規劃三類子問題，依據子集合選擇與路徑規劃的特性使用不同演算法策略，並將策略分配至各階段對應不同子問題，以有效降低搜尋空間，提升求解效率與解的品質。
3. 建構符合實務規則之數學模型，納入物流公司既有主線結構、車輛容量與時間窗限制，使得到結果能直接應用於實務配送規劃，同時兼顧理論與實務規則，亦可作為後續發展配送決策與智慧化物流調度方法之基礎，提供一個具實務參考價值之研究。

本研究提出分階段路線規劃架構，將物流公司實務規劃規則納入模型設計，以降低搜尋空間並提升解的品質。本研究不僅驗證固定主線條件下路徑最佳化之可行性外，亦可作為後續發展配送決策與智慧化物流調度方法之基礎，提供一個具實務參考價值之研究。

### 1.3 章節規劃

本論文分為五章，第二章為背景知識與文獻探討，介紹物流路線規劃的相關應用。第三章為研究方法，針對本研究的研究架構、流程處理進行說明。第四章為展示實驗結果與分析結果。第五章為說明研究結論與未來研究方向。

## 第二章 背景知識與文獻探討

本章介紹本研究所需之背景知識與相關文獻，內容包含車輛路徑問題(Vehicle Routing Problem, VRP)[1]、基因演算法(Genetic Algorithm, GA)[3]與蟻群演算法(Ant Colony Optimization, ACO)[4]的理論基礎及研究發展，同時亦探討與本研究主題相關之文獻成果，並對其研究結果進行比較與分析。

### 2.1 Vehicle Routing Problem

VRP 屬於典型 NP-hard 組合最佳化問題，旨在為同一組同質車輛，依據顧客的需求量，分配並規劃配送路線的問題，並在滿足各種限制條件下，最小化成本函數。隨著應用需求的多樣化，VRP 根據不同的限制條件衍伸出多種的變形，常見的類型包含容量限制車輛路徑問題(Capacitated Vehicle Routing Problem, CVRP)[6]、時間窗車輛路徑問題(Vehicle Routing Problem with Time Windows, VRPTW)[7]、兼具自有與委外之車輛路徑問題(Vehicle Routing Problem with Private Fleet and Common Carrier, VRPPC)[8]。而在基本 VRP 模型中，須滿足以下基本限制條件：

1. 每輛車必須從配送中心出發並返回配送中心。
2. 每個顧客僅允許被拜訪一次且僅能由一輛車負責配送。
3. 每個顧客都必須完成配送。

#### 2.1.1 Capacitated Vehicle Routing Problem

CVRP 是 VRP 中最基本且最廣泛應用的變體之一。該問題描述一組具有相同容量限制之同質車輛，需自單一配送中心出發，對一組已知需求之顧客進行配送服務。每位顧客具有確定的需求量，而每輛車的總載運量不得超過其最大容量限制。在此條件下，決策者需同時規劃顧客的分配以及服務順序，使所有顧客皆能被滿足，且每輛車最終返回配送中心。由於該問題同時涉及路徑規劃與容量配置兩種決策，其本質上結合旅行推銷員問題(Traveling Salesman Problem, TSP)之路徑排序特性與裝箱問題(Bin Packing



Problem, BPP)之容量分配特性，屬於 NP-hard 問題，因此 CVRP 在理論與實務上皆具重要研究價值，並廣泛應用於物流與運輸規劃領域。

在相關研究中，Chen 等人提出疊代變數鄰域下降法(Iterated Variable Neighborhood Descent, IVND)[9]以最佳化配送路徑。該方法以變數鄰域下降法(Variable Neighborhood Descent, VND)[10]作為局部搜尋基礎，透過 relocation、swap、2-opt 及 2-opt\*等運算子，在路線內及路線間調整顧客分配，並引入擾動機制(Perturbation)，以降低總運輸成本。

另一方面，Alesiani 等人的研究提出 CC-CVRS(Constrained Clustering Capacitated Vehicle Routing Solver)[11]，利用具約束分群演算法(Constrained Clustering Algorithm)解決 CVRP。該方法旨在解決大規模實例中客戶數量龐大、傳統求解方法計算成本過高的問題。首先根據客戶的地理位置與需求量將其劃分為滿足車輛容量限制的多個族群，並對每個族群進行配送路徑規劃，以同時保留容量約束並縮短計算時間。分群所得的初步解可作為後續啟發式或精確演算法的初始解，以提升整體求解效率與路徑品質。

### 2.1.2 Vehicle Routing Problem with Time Windows

相較於僅考量容量限制的 CVRP，VRPTW 在考量容量限制外引入時間窗限制條件。每個顧客節點必須在規定的時間區間內接受服務。車輛不得早於最早開始時間進行服務，且不得晚於最晚結束時間，確保在規定時間內完成顧客節點的服務，以最小化總行駛距離為目標。VRPTW 在學術研究與實務應用中均被廣泛探討，現有文獻已提出多種演算法以解決此最佳化問題。

在相關研究中，Ombuk 等人提出多目標基因演算法(Multi-Objective Genetic Algorithm, MOGA)[12]，同時考量最小化車輛數與總行駛距離兩項目標，並採用 Pareto 排序機制進行族群選擇，以維持解之多樣性與非支配解特性。在基因操作方面，該研究比較多種交配機制，並提出最佳成本路徑交配(Best Cost Route Crossover, BCRC)作為問題導向之交配運算子。研究結果顯示，相較於傳統排列式交配方法，BCRC 透過路徑層級重組與基於成本之重新插入機制，可同時改善解的可行性與品質，並有效降低車輛數



與總行駛距離，進而提升整體搜尋效能。

另一方面，Maroof 等人提出 HGA-SIH(Hybrid Genetic Algorithm-Solomon Insertion Heuristic)混合式基因演算法[13]，透過結合傳統隨機生成機制與 SIH(Solomon Insertion Heuristic)，以提升初始解的品質與多樣性，並促進演算法更有效地朝向全域最佳解收斂。此外，該研究採用整數編碼表示染色體，並透過特定的解碼機制，將染色體轉換為符合容量與時間窗限制的可行路徑。同時，SIH 亦被整合為局部搜尋算子，以進一步強化演化過程中解的品質與收斂能力，並於 Solomon benchmark[14]問題上進行驗證，結果顯示該方法具有良好之求解能力。

此外，Yu 等人的研究[15]提出一種混合演算法，結合 ACO 與禁忌搜尋演算法(Tabu Search, TS)，並在 ACO 中引入鄰域搜尋以改善解品質。Zhang 與 Li 等人則提出混合啟發式和諧搜尋(Hybrid Heuristic Harmony Search Algorithm, HHHSA)[16]，結合局部搜尋以最佳化解空間探索。兩篇研究於 Solomon 測試實例的實驗結果顯示，所提出的方法在保持計算效率的同時，亦能產生高品質解，顯示啟發式與混合啟發式演算法在現代物流配送與路線最佳化中的有效性。

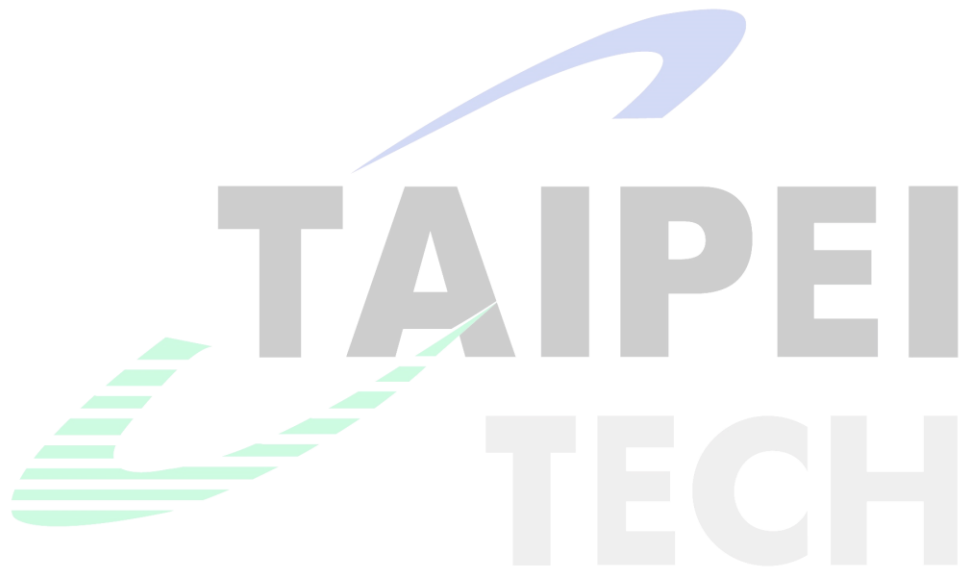
### **2.1.3 Vehicle Routing Problem with Private Fleet and Common Carrier**

相較於假設擁有足夠自有車輛以服務所有顧客的 VRP，VRPPC 在既有的自有車輛 (Private Fleet)容量限制外，引入了外部承運商(Common Carriers)作為支援。在此架構中，當自有車隊(Private Fleet)無法滿足所有需求，或是服務特定顧客的成本過高時，可選擇將部分訂單外包給外部物流業者。決策過程需同時權衡固定與變動成本，以及外部物流業者的外包成本，確保在滿足所有顧客需求的前提下，合理分配訂單至自有車隊或外部承運商，以最小化總物流成本。

在 Côté 等人[17]的研究中，針對 VRPPC 提出一種基於禁忌搜尋(Tabu Search, TS)之

啟發式演算法。該方法允許顧客在自有車隊配送與外包服務之間進行指派決策，並以最小化整體成本為目標，成本包含車輛固定成本、行駛距離成本以及外包服務所產生之外包成本。此外，其方法結合顧客插入與交換等鄰域搜尋策略，並透過自適應懲罰機制處理容量限制問題，以提升解的品質與搜尋效率。

此外，Bolduc 等人亦提出擾動元啟發式演算法(Perturbation Metaheuristic)[18]以解決VRPPC。該方法引入外包決策評估機制，於建構與擾動解的過程中評估顧客節點由自有車隊或委外車輛服務之效益。透過權衡啟用自有車輛的高昂固定成本與委外配送的變動成本，該方法能精確界定自有車隊的服務邊界，以達成包含外包費用在內的總物流成本最小化。



## 2.2 Genetic Algorithm

基因演算法(Genetic Algorithm, GA)由 John Holland[3]提出，其靈感來自達爾文的進化論，模擬了適者生存及基因傳遞的概念，已被廣泛應用於各類最佳化問題[19-21]。GA 為一種基於自然選擇與遺傳機制的元啟發式演算法，用於解決組合最佳化問題。每個候選解稱為個體(Individual)，由染色體(Chromosome)表示，染色體由多個基因(Gene)組成。演化過程中，族群依據適應值進行選擇(Selection)，並透過交配(Crossover)交換基因資訊，以及突變(Mutation)隨機改變部分基因，以維持族群多樣性並避免陷入局部最佳解。

以多維度背包問題(Multidimensional Knapsack Problem, MKP)為例，Beasley 等人[22]提出了一種結合問題特定知識的遺傳演算法框架。透過引入基於線性規劃鬆弛對偶變數的虛擬效用比率，並根據此設計雙階段貪婪修復運算子 (DROP/ADD Repair Operator)。其實驗結果顯示，此種將搜尋限制於可行解空間並結合啟發式修復的策略，能在合理時間內產出高品質解。

針對有群組結構特性之最佳化問題，分群基因演算法(Grouping Genetic Algorithm, GGA)[23]進一步將群組作為染色體之基本建構單元(building blocks)，並使交配與突變等操作亦以群組為單位進行調整，從而使演化過程得以直接反映分群結構之變化。Bennani 等人[24]則進一步提出結合群集分裂、群集合併、區域搜尋及島嶼模型(Island Model)之 GGA 方法，以提升搜尋能力與族群多樣性，並可於演化過程中自動決定最佳群數，於多種資料集上取得優於 K-means 與 DBSCAN 之分群品質與穩定性。

然而，當問題進一步擴展至高維度且具明顯子問題相依性時，單一族群之演化架構仍可能面臨搜尋效率不足之問題。因此，後續研究發展出合作式共同演化基因演算法(Cooperative Co-evolutionary Genetic Algorithm, CCGA)，透過分而治之的概念將原始問題分解為多個子問題，並以多子族群分別進行演化搜尋，再透過協同機制組合各子族群解，以提升大型問題之整體求解效率與解品質。

## 2.2.2 Cooperative Co-evolutionary Genetic Algorithm

針對高維度且具子問題相依性之最佳化問題，合作式共同演化基因演算法 (Cooperative Co-evolutionary Genetic Algorithm, CCGA)[25] 由合作式共同演化演算法 (Cooperative Co-evolutionary Algorithm, CCEA)[26] 概念延伸而來，其結合 GA 與共同演化 (Co-evolution) 機制，並採用分而治之 (Divide-and-Conquer) 之概念，將原始複雜問題分解為多個子問題，分別由不同子族群 (subpopulations) 分別進行演化搜尋，再透過合作機制組合各子族群之代表個體形成完整解，以提升大型問題之搜尋效率與整體解品質。

Adedigba 等人[27] 提出一種應用於多機器人任務排程問題之 CCGA，並以南極區域探測任務作為研究背景。該研究指出，當任務點數量增加且空間分布廣泛時，傳統方法容易因搜尋空間過大而降低搜尋效率。因此，研究中採用 iVec (iterative Vector Clustering) 機制，根據任務點相對於共同場站 (Common Depot) 之距離與方位角進行空間分群，並將各群組分配至不同子族群進行獨立演化搜尋。此外，於適應值評估階段，演算法會結合其他子族群之代表性個體進行協同評估，以計算完整解之整體成本。

## 2.3 Ant Colony Optimization

蟻群演算法(Ant Colony Optimization, ACO)為 Marco Dorigo[4]提出的元啟發式演算法，透過模擬螞蟻在覓食過程中釋放費洛蒙尋找最短路徑的行為來解決組合最佳化問題。在解之建構過程中，每隻螞蟻以輪盤選擇法選擇下一個節點，選擇機率根據當前路徑的費洛蒙濃度與啟發式信息決定，此決策機制屬於序列式決策(sequential decision)，使其特別適用於路徑規劃問題。當螞蟻完成路徑構建後，費洛蒙將依據解的品質進行更新，以提高優良路徑被選擇的機率，並透過費洛蒙蒸發機制避免過度收斂。上述特性使得 ACO 在處理路徑相關問題時具有良好的彈性與效率，並可延伸應用於具複雜限制條件之最佳化問題。

在 ACO 延伸架構方面，Gambardella 等人提出 MACS-VRPTW(Multiple Ant Colony System for Vehicle Routing Problems with Time Windows)[28]，採用雙蟻群協作架構，分別最佳化車輛數量與總行駛距離，並透過費洛蒙資訊交換實現多目標協同搜尋。Bell 等人[29]同樣將多蟻群架構應用於 VRP，透過為各配送車輛設置獨立費洛蒙，降低多車路線規劃時的相互干擾，並結合 2-opt 局部搜尋、區域更新及候選名單機制，以提升大規模問題之解品質與求解效率。

此外，在不確定環境下之應用方面，Di Caprio 等人[30]提出一種針對不確定環境之最短路徑問題的模糊蟻群最佳化演算法。該方法結合 ACO 並採用梯形與常態模糊數刻畫邊權，透過  $\alpha$ -cut 方法近似計算混合模糊權重之路徑成本。實驗結果顯示，相較於 GA、粒子群最佳化(Particle Swarm Optimization, PSO)與人工蜂群演算法(Artificial Bee Colony, ABC)，在收斂速度、迭代次數及整體運算時間上均具有明顯優勢，特別是在高複雜度問題中的表現。

### 2.3.1 Hybrid Ant Colony Optimization

混合式元啟發式旨在解決單一元啟發式演算法在複雜組合最佳化問題中難以同時兼顧全域探索能力與區域開發效率的問題，透過結合不同演算法或啟發式策略的互補特性，以提升搜尋效能與解品質穩定性。在此背景下，混合式蟻群演算法(Hybrid Ant Colony Optimization, HACO)作為對 ACO 的改良方法，主要針對其正回饋機制導致過早收斂，以及區域改良能力不足等問題進行改善。HACO 結合 ACO 與其他啟發式策略，使 ACO 負責全域搜尋，輔助方法則用於初始解強化、區域改良與維持解多樣性。

為進一步提升 ACO 之搜尋效能，Ding 等人[31]提出結合區域搜尋(Local Search, LS)之混合式蟻群演算法，透過調整費洛蒙更新策略，引入災變操作(Disaster Operator)以降低搜尋過程陷入區域最佳解之風險並維持搜尋多樣性。同時亦考量候選清單(Candidate List)機制，以限制螞蟻在解建構過程中的選擇範圍，進而提升演算法的收斂速度。

在車輛路徑問題應用方面，Molina 等人[32]針對異質車輛途程問題並考慮時間窗限制(Heterogeneous Vehicle Routing Problem with Time Windows, HVRPTW)，提出結合 ACS 與 Memetic Algorithm 之混合式方法。該方法利用 ACS 進行全域解建構，並結合變動鄰域禁忌搜尋(Variable Neighborhood Tabu Search, VNTS)進行區域搜尋，以提升解品質。同時引入 Holding List 機制以管理未服務顧客。實驗結果顯示，該方法於多組基準測試中可有效降低總配送成本並提升收斂表現。

另一方面，Wang 等人[33]提出一種結合共生生物搜尋(Symbiotic Organisms Search, SOS)與 ACO 的混合演算法 SOS-ACO(Hybrid Symbiotic Organisms Search and Ant Colony Optimization Algorithm)，應用於旅行推銷員問題 TSP。方法利用 SOS 對 ACO 參數進行自適應調整，亦結合局部最佳化策略以提升收斂速度。並於 TSPLIB 中多組 TSP 測試實例進行驗證。實驗結果顯示，相較於既有方法，在相同條件下 SOS-ACO 能取得較佳解，且對參數變動具有良好的自適應能力。



## 2.4 現有方法之限制與比較探討

本節旨在探討現有 VRP 及其演算法在實務物流配送中的適用性與限制。VRP 及其延伸模型雖已在學術上有廣泛研究，多以全域最佳化為核心，但對實務限制之考量仍相對不足，導致理論最佳解在實務中難以應用。此外，配送任務涉及多重限制且包含不同層次的決策目標，單一演算法難以同時兼顧各階段需求，因此需依階段特性採用不同演算法以充分發揮各方法優勢。本節將分析現有方法與實務需求之差異，並為後續多階段演算法設計提供理論基礎。

### 2.4.1 現有 VRP 模型與實務需求之差異

傳統 VRP 及其延伸模型，多假設配送需求於規劃階段即已完整已知，並允許在滿足限制條件的前提下，對所有配送路線進行全域性重新編排。其中，VRPTW 與 VRPPC 等典型 VRP 延伸模型多以全域路線重構或全域決策為核心假設，透過重新決策所有客戶之服務順序與路線配置，以達成整體成本最小化。此外，VRPTW 通常允許車輛於時間窗開始前提早抵達顧客位置，並透過等待機制滿足服務時間限制。然而，在本研究所探討之實務配送情境中，配送車輛必須於時間窗規定之期間內抵達並完成服務，不允許透過提前抵達後等待之方式調整時程，使得可行解空間進一步受到壓縮，亦增加路線規劃之困難度。

有別於上述全域性重構之方式，本研究問題受既有路線結構限制，其核心挑戰並非單純針對所有路線進行最佳化，而是如何在既有配送架構下進行有限度調整。實務上，運務士長期負責固定之主線配送路線，並依循主線既定順序服務特定店鋪，此類主線多由歷史經驗逐步累積而成，雖未必符合數學最佳化結果，但因長期固定使用而具有高度穩定性與慣性。若演算法允許進行全域探索以重組配送路線，即使結果在理論上符合所有限制條件，仍可能因違背運務士既有習慣而被視為不可接受。

現有文獻中已有多種 VRP 等相關研究，但多數模型聚焦於以數學模型描述成本結構並進行目標函數最佳化，與實務決策流程仍存在落差。本研究，物流公司傾向優先消化既有主線之剩餘運能，僅在確定無法於內部調整下達成可行解時，才啟用高成本之支援線。此種具明確決策順序與成本階層之處理邏輯，並未被多數 VRP 模型所明確反映。

另一方面，VRPTW 通常允許車輛於顧客時間窗開始前提早抵達，並透過等待機制滿足後續服務時間限制。然而，在本研究之實務配送場域中，配送車輛不得提早抵達店鋪，需於指定時段內準時完成配送服務，因此無法透過等待時間調整路線時程。相較於 VRPTW，此限制進一步壓縮可行解空間，並提高路線規劃與時間協調之困難度，使部分 VRPTW 模型與相關啟發式方法於本研究情境下之適用性受到限制。

在實務配送流程中，物流公司常依循既有之人工編排流程進行路線調整，基於經驗法則逐步篩選可行組合。如依主線是否超載挑選候選店鋪、依行政區域與距離物流中心遠近對店鋪進行分區、限制路線方向與路線總工時上限等。此類流程雖可確保實務可行性，但其高度仰賴資深人員的判斷，特別是在面對多條主線同時超載而需進行額外規劃之情境時，人工決策難以在龐大的解空間中進行有效的全局評估，往往僅能針對個別路線進行局部調整，且需耗費大量時間成本。相較於演算法可於短時間內產生品質較佳之解，現行人工編排流程於營運效率與成本控制方面仍具有顯著改善空間。

綜上所述，現有 VRP 模型在反映既有主線配送路線及其店鋪服務順序不可任意變動之實務限制、人員接受度與具成本階層之調度決策等實務特性方面仍顯不足。基於此，本研究旨在提出一套兼顧實務可行性與計算效率之演算法架構，以作為現行人工編排流程與 VRP 模型之間的橋樑。



## 2.4.2 單一演算法局限性與分階段設計

針對 VRP 及其變體中，搜尋空間隨節點數呈階乘式增長，單一階段的全域搜尋演算法在計算效率與解品質上皆面臨明顯侷限。為解決此問題，近年研究多採用分階段求解，透過將原問題進行分解，拆解為多個子問題，以有效縮減解空間並提升求解效率。

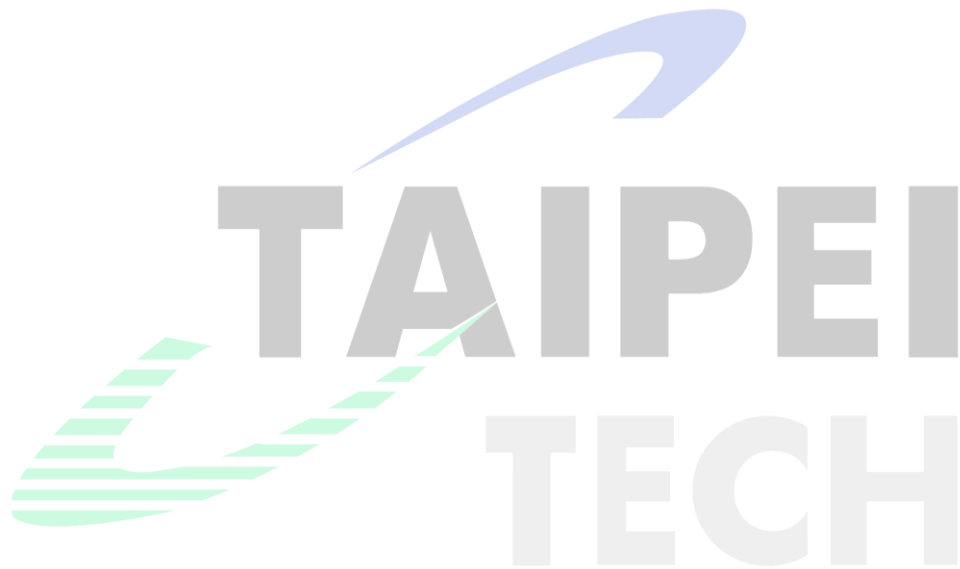
此類分階段與分解策略並不限於 VRP 領域，在其他組合最佳化問題中亦被證實具備良好效果。Ma 等人[34]針對具有指數級搜尋空間之高維度特徵選擇問題，提出兩階段混合蟻群演算法(Two-stage Hybrid Ant Colony Optimization for High-Dimensional Feature Selection, TSHFS-ACO)。該方法首先透過區間式啟發機制對解空間進行初步限縮，以降低問題維度並界定搜尋範圍，其後再由 ACO 於特定子空間中進行細部搜尋。實驗結果顯示，此種方法能有效緩解大規模組合最佳化問題中的維度災難(Curse of Dimensionality)，並提升整體求解效率與全域收斂能力。

在 VRP 相關研究中，分階段與分解策略同樣被廣泛應用。Frias 等人[35]針對能源最小化車輛路徑問題(Energy Minimizing Vehicle Routing Problem, EMVRP)提出一種兩階段的混合式演算法架構(Two-phase Hybrid Algorithm)。該方法利用 K-Means 或 K-Medoids 分群演算法，將顧客節點劃分為多個子集合，以降低問題規模並引導後續搜尋方向，接著再透過改良式 ACO 進行路徑建構與最佳化。該方法在啟發式資訊中同時考慮距離、能源成本，並結合變鄰域搜尋(Variable Neighborhood Search, VNS)進行區域最佳化，改善 ACO 易陷入區域最佳解的問題。實驗結果顯示，採用分層式架構能有效降低複雜問題的搜尋難度，並提升整體解品質。

此外，Jiao 等人[36]針對具能源限制之車輛途程問題(Vehicle Routing Problem with Energy Constraint, VRPEC)，提出一種基於任務分群的多階段啟發式演算法(Multi-Stage Vehicle Routing Algorithm based on Task Grouping, MSVR-TG)。該方法透過任務分群機制對原始問題進行問題分解(decomposition)，將其劃分為多個較易處理之子問題，以降低

整體問題複雜度並提升求解效率；後續再透過序列規劃與路徑調整逐步優化解品質。實驗結果顯示，此種基於分解之策略在解的品質與計算效率上皆具有良好表現。

綜上所述，當 VRP 規模隨節點數增加而迅速擴張時，相較於單一演算法直接進行全域搜尋，分階段設計可透過問題結構化與搜尋空間逐步縮減，更有效地平衡計算效率與解品質。基於此，本研究延續相關文獻之設計理念，採用分階段求解架構，並結合物流公司之實務規劃流程，將原始問題拆解為明確功能之子階段，於各階段分別引入適切之演算法機制，以在可接受的計算時間內獲得穩定且高品質之解。



## 第三章 研究方法

本章說明本研究所採用的研究方法，並對研究問題進行分析與描述，同時說明系統架構，內容包含各模組之間的功能與關係。

### 3.1 問題實例描述

本研究使用由物流公司提供的初始路線資料，該資料為物流公司依據以往配送經驗所規劃的主線配送路線，即車輛行駛路線與配送順序的初步規劃。本研究旨在解決在已有主線配送、材積量以及時間窗限制的條件下，重新規劃支援線的配送問題。由於每日各店鋪的實際配送需求可能變動，因此當主線配送車輛無法滿足所有需求，便需要調整配送策略，以避免延誤且提升整體效率。

為提升配送規劃的效率並滿足需求，本研究發展一套配送方法，在不更動主線配送路線順序的前提下，針對超過材積量限制的路線進行動態調整，將其線路上的店鋪重新分配至其他可用車輛或額外支援車輛，藉此平衡車輛載運材積量與配送時間的限制，提升整體配送效能，同時有效減少配送車數量，進而降低整體運輸成本。

在配送過程中，本研究根據「A 物流公司」路線規劃資料，設置限制條件以確保配送過程順利進行。條件如下：

1. 配送車貨物裝載率上限為 100%。
2. 店鋪配送需要符合時間窗限制，到店時間應於表定前一小時至後三十分鐘內。
3. 車輛配送路線總配送時長不能超過五小時。
4. 主線可以支援運送支援候選店鋪，但原店鋪配送順序不能調整。
5. 路線規劃維持主線路徑結構，並盡量減少店鋪異動數量，以符合配送人員操作習慣
6. 路線規劃依照地理位置分群，並將店鋪分為近、中和遠區，近區與中區店鋪無需考慮方向問題，遠區店鋪若第一間店鋪已挑選物流中心以北的店鋪，後續不得挑選物流中心以南的店鋪，東西向亦然。

透過對歷史路線資料的整理與分析及路線規劃過程中的實務經驗。本研究歸納以下規劃規則：

1. 判斷所有主線的總材積量是否超過車輛可承載上限，若未超出，則忽略該條主線並進入下一條主線進行判斷；若超出，則對該主線進行店鋪抽取，將挑選出的店鋪作為候選店鋪，讓主線材積量降至可接受範圍。
2. 彙整所有被抽出的候選店鋪，依照地理位置進行分區，以利後續支援線編排的區域性規劃。
3. 嘗試將候選店鋪重新分配至其他仍具剩餘容量之主線，並同時檢查車輛載重限制與時間窗限制是否滿足，同時評估加入該店鋪後的整體路線時間與物流效率。
4. 無法成功分配回主線的剩餘候選店鋪，則額外規劃新的支援線。依據與物流中心的距離劃分為近區、中區與遠區。在近區與中區內規劃支援線時可不考慮方向一致性。在遠區需確保所有店鋪方位一致，避免繞行或折返，同時須控制路線時間不超過五小時且每間店鋪的到店時間落在其允許時間窗內。
5. 為確保規劃結果之實務可行性，本研究於路線規劃過程中即透過 Open Source Routing Machine (OSRM)進行路網距離與行駛時間之計算，以作為演算法評估與路線構建之基礎。在此基礎上，進一步採用 Google Maps Routes API 針對已完成之主線與支援線路線進行重新估算，以反映實際道路條件下之行駛距離與配送時間。最後，將更新後之配送順序、預估時間與距離資訊回寫至原始路線資料中，作為後續分析與物流驗證之依據。

### 3.1.1 符號定義

#### 變量定義

$r_k$ : 第  $k$  條路線

$Q_k$ : 車輛  $k$  的材積量上限

$Q_{sup}$ : 支援線的材積量上限

$L_k$ : 車輛  $k$  總材積量

$v_i$ : 店舖  $i$  所需材積量

$T_{max}$ : 車輛配送時長上限

$T_k$ : 車輛  $k$  總配送時長

$d_{0,j}$ : 物流中心到店舖  $j$  之間距離

$d_{i,j}$ : 店舖  $i$  到店舖  $j$  之間距離

$s_i$ : 店舖  $i$  之服務時間

$t_{i,j}$ : 店舖  $i$  到店舖  $j$  花費時間

$e_i$ : 店舖  $i$  最早開始服務時間

$l_i$ : 店舖  $i$  最晚開始服務時間

$C$ : 車輛固定運輸成本

#### 集合定義

$M$ : 所有主線集合

$H$ : 所有支援線集合

$N$ : 所有店舖集合

$K$ : 所有車輛集合

$R$ : 候選店舖集合

$R_{near}$ : 近區候選店舖集合

$R_{mid}$ : 中區候選店舖集合

$R_{far}$ : 遠區候選店舖集合

### 基因演算法參數定義

$P_k$ : 第  $k$  個子族群

$P_k^g$ : 第  $g$  世代中第  $k$  個子族群

$N_{sub}$ : 子族群大小

$E$ : 菁英保留數量

$G$ : 最大世代數

$F_{k,n}^g$ : 第  $g$  世代中第  $k$  個子族群之第  $n$  個個體的適應值

$p_{k,n}^g$ : 第  $g$  世代中第  $k$  個子族群之第  $n$  個個體被選中的機率

$Z_{k,n}^g$ : 第  $g$  世代中第  $k$  個子族群之第  $n$  個個體

$p_c$ : 交配機率

$p_m$ : 突變機率

$F$ : 適應函數

$w_1$ : 車輛懲罰係數

$w_2$ : 店舖數量懲罰係數

$w_i$ : 店舖 i 權重

$\theta_i$ : 店舖 i 之需求分級係數

### 蟻群演算法參數定義

$\alpha$ : 費洛蒙重要度

$\beta$ : 啟發函數重要度

$\rho$ : 費洛蒙蒸發速率

$m$ : 螞蟻數量

$\tau_0$ : 初始費洛蒙濃度

$\tau_{i,j}$ : 初始費洛蒙濃度

$\Delta\tau_{i,j}^{iter}$ : 當前迭代最佳路徑的費洛蒙濃度增量

$\eta_{i,j,u,k}$ : 將店舖 u 插入至路線 k 中店舖 i 與店舖 j 之間位置之啟發式資訊

$\eta_{i,j}$ : 距離倒數啟發式資訊

$p_{i,j,u,k}^a$ : 螞蟻 a 將店舖 u 插入路線 k 中店舖 i 與店舖 j 之間位置的轉移機率

$p_{i,j}^a$ : 螞蟻 a 從店舖 i 移動到店舖 j 的轉移機率

$N_i^a$ : 螞蟻 a 在店舖 i 可選擇的可行鄰域集合

$N_k$ : 第 k 個鄰域結構

$\Phi_{i,j}$ : 店舖 i 至店舖 j 之移動成本

$q$ : 控制貪婪演算法之隨機變數

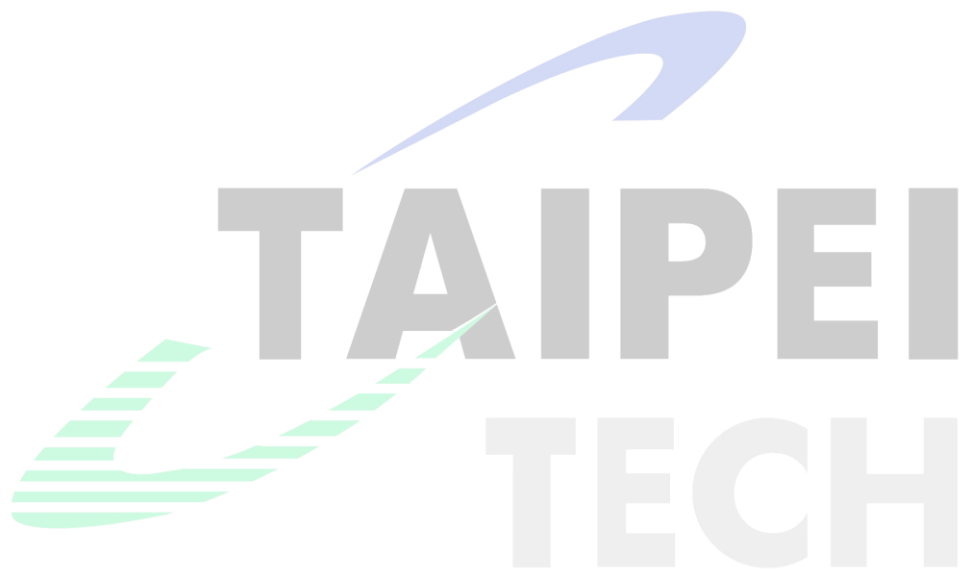
$q_0$ : 控制貪婪演算法與隨機比例之閾值



## 決策變數

$x_{i,j}^k \in \{0,1\}$ : 車輛  $k$  是否從店舖  $i$  行駛到店舖  $j$

$z_{k,n,i}^g \in \{0,1\}$ : 第  $g$  世代中第  $k$  個子族群內第  $n$  個個體之第  $i$  個店舖抽取決策



## 約束條件

在配送路線規劃中，為確保配送過程順利並符合實務需求，須滿足多項限制條件。

式(3.1)表示每間店鋪之限制，確保所有店鋪皆必須且只能由一輛車輛拜訪一次。

$$\sum_{k \in K} \sum_{i \in N} x_{i,j}^k = 1, \quad \forall j \in N \quad (3.1)$$

式(3.2)與式(3.3)分別為配送中心之出發與返回限制，確保每輛車皆由配送中心出發，且在完成配送任務後必須返回配送中心。

$$\sum_{j \in N} x_{0,j}^k = 1, \quad \forall k \in K \quad (3.2)$$

$$\sum_{i \in N} x_{i,0}^k = 1, \quad \forall k \in K \quad (3.3)$$

式(3.4)用於車輛材積量限制，規定每輛配送車的總材積量不得超過該車輛的材積量上限。

$$\sum_{i \in r_k} v_i \leq Q_k, \quad \forall k \in K \quad (3.4)$$

式(3.5)為店鋪節點到達時間限制，確保配送車在完成前一店鋪的服務後，經過兩點間的行駛時間才抵達下一節點。

$$t_j \geq t_i + s_i + t_{i,j}, \quad \forall i, j \in N \quad (3.5)$$

式(3.6)為時間窗限制，規定每家店鋪的到達時間必須落在允許的服務時間範圍內，本研究設定為表定時間的前一小時至後三十分鐘內。

$$e_j \leq t_j \leq l_j, \quad \forall j \in N \quad (3.6)$$

式(3.7)車輛總配送時長限制，要求每輛配送車從出發至返回的總配送時長不得超過最大允許值，設定最大值為五小時。

$$T_k \leq T_{max}, \quad \forall k \in K \quad (3.7)$$

式(3.8)、式(3.9)和式(3.10)則根據配送店舖與物流中心間之距離，將候選店舖劃分為近區、中區與遠區。近區代表距離物流中心三公里以內的區域；中區為距離三至五公里之間的區域；遠區則為距離超過五公里的區域。

$$R_{near} \Rightarrow d_{0,j} \leq 3, \quad \forall j \in N \quad (3.8)$$

$$R_{mid} \Rightarrow 3 < d_{0,j} \leq 5, \quad \forall j \in N \quad (3.9)$$

$$R_{far} \Rightarrow d_{0,j} > 5, \quad \forall j \in N \quad (3.10)$$

### 目標函數

本研究針對物流公司之配送路徑規劃問題進行探討，建構一個以實務營運效益為導向之最佳化模型。具體而言，本研究採用階層式目標(Hierarchical Objective)，將多個目標以分層方式進行最佳化，優先最小化支援線所需之車輛數量，再進一步最小化配送車輛之總行駛距離，以兼顧整體配送效益。本研究將上述兩項子目標整合為一字典序最佳化(Lexicographical Optimization)模型。在給定可行解空間的前提下，整體最佳化數學模型公式如式(3.11)所示。

$$\text{Lex-Min} (f_{vehicle}(S), f_{travel}(S)) \quad (3.11)$$

本研究採用階層式目標，將多個物流指標以分層方式進行最佳化，同時考量下列兩項目標的最小化。於此架構下，各目標依序進行評估，先尋求主要目標之最佳解，再於其條件下進一步最佳化次要目標，以兼顧整體配送效益。其中式(3.12)代表支援車所需之車輛數量，式(3.13)代表總行駛距離。

$$f_{vehicle}(S) = \sum_{h \in H} \sum_{\substack{j \in N \\ (j \neq 0)}} x_{0,j}^h \quad (3.12)$$

$$f_{travel}(S) = \sum_{k \in K} \sum_{i \in N} \sum_{\substack{j \in N \\ (j \neq i)}} d_{i,j} \cdot x_{i,j}^k \quad (3.13)$$

## 3.2 系統架構

本研究提出的方法其架構如圖 1 所示，針對物流公司之配送路徑規劃問題，建構四個模組，包含資料模組(Data Module)、路線模組(Route Module)、API 模組(API Module)與評估模組(Evaluation Module)。首先，讀取原始配送路線資料(Original Route Data)，透過資料模組轉換為後續路線模組可以處理的資料格式，使其符合路線模組所需之輸入格式，同時使用 Open Source Routing Machine(OSRM)建立配送店鋪之間的距離矩陣，提供路線規劃所需之行駛距離資訊。在路線模組中，透過店鋪抽取(Store Extraction)將超出承載限制之主線店鋪進行篩選，並形成候選店鋪集合，後續再根據候選店鋪分配(Store Assignment)和支援線規劃(Support Route Planning)將候選店鋪重新分配至主線或支援線並結合 Google Maps Platform 提供的 Routes API 估算實際行車時間。完成路線規劃後，路線評估(Routes Evaluation)對規劃結果進行分析與比較，最終輸出最佳化路線結果(Optimized Route Data)，提供作為實務規劃的參考。該架構旨在模擬實際配送情境，並作為實作方法之基礎。後續將針對各模組與相關資料設定進行說明。

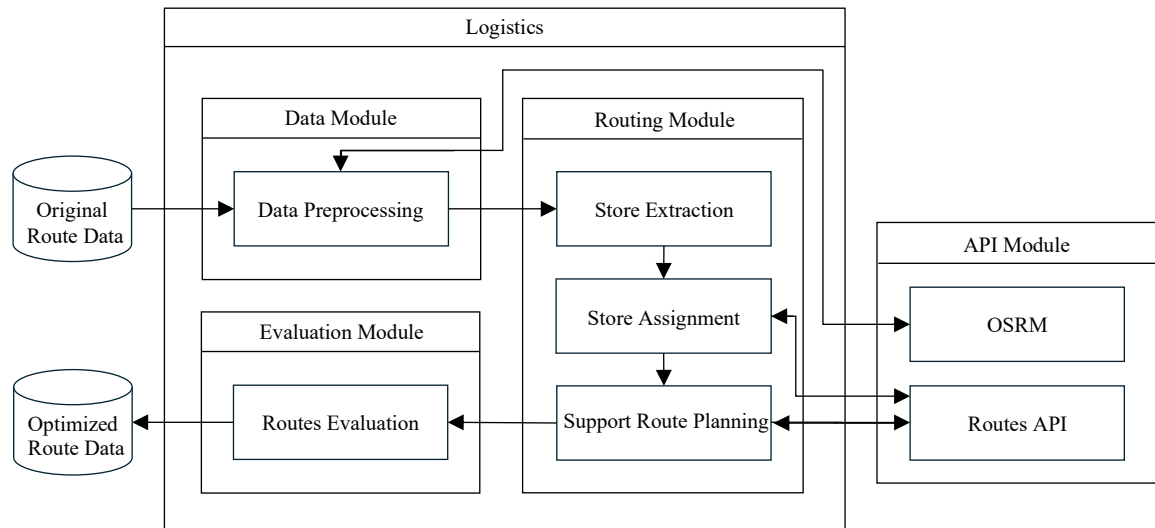


圖 1 系統架構圖

### 3.2.1 原始配送路線資料 (Original Route Data)

本研究系統的原始配送路線資料包含路網資料、滯店時間、店鋪主檔以及原始路線資料。路網資料記錄各店鋪之間在不同時段的路程時間，包括依據原始地圖資料計算的路程時間及實際計算的平均行駛時間。滯店時間表示車輛在各店鋪進行服務所需的停留時間。店鋪主檔記錄店鋪的基本資訊，包括店鋪編號、名稱、地址以及經緯度等基本店鋪資訊。原始路線資料則呈現最初規劃好的主線資料，內容涵蓋各車次及其對應的店鋪、表定到店時間，以及車輛的材積量與裝載率，以利後續路線規劃與配送。

### 3.2.2 資料模組 (Data Module)

資料模組主要負責整合歷史配送資料、店鋪資訊與路網資料，並將原始配送資料轉換為後續演算法模組可使用之標準化資料格式。由於原始資料來源眾多，且不同資料表間之欄位格式存在差異，因此需透過資料清理、欄位轉換與結構化處理，以建立一致化之配送資料模型。處理內容包含車次與店鋪資訊整合、時間資料轉換、店鋪需求量與車輛容量資訊建立，以及路網時間矩陣建構等。經處理後之資料，將作為後續路線規劃、可行性檢查與演算法評估之基礎。

#### 3.2.2.1 資料前處理 (Data Preprocessing)

資料前處理主要針對原始配送資料進行清理、轉換與整合，以建立後續路線規劃模組可直接使用之標準化資料格式。對於缺失、不完整或異常之資料，系統則透過補值、格式修正等方式進行處理，以提升資料品質與後續求解穩定性。

在路網資料處理方面，本研究透過開放原始碼路由服務系統(Open Source Routing Machine, OSRM)建立店鋪間之行駛時間矩陣與距離矩陣。系統根據店鋪經緯度資訊，結合實際道路進行最短路徑與行駛時間計算，以取得各店鋪間之配送移動時間與距離資訊。相較於僅使用歐式距離進行估算，OSRM 能更真實反映道路結構與實際配送情境，作為

後續路線規劃、配送成本計算與時間檢查之基礎。透過上述資料整合與前處理，可建立符合實務配送情境之資料結構，供後續模組進行路線建構、可行性驗證與成本評估。

### 3.2.3 路線模組 (Route Module)

路線模組主要負責針對既有配送路線進行檢查、調整與最佳化，以確保在滿足車輛容量與時間限制之下，提升整體配送效率。本研究將固定主線配送問題形式化為三個子問題，分別對應於店鋪抽取、候選店鋪再分配及支援線規劃，並依序處理超載路線，重新分配候選支援店鋪，並針對無法分配到主線的店鋪額外規劃支援路線。

#### 3.2.3.1 店鋪抽取 (Store Extraction)

店鋪抽取針對主線進行材積量檢查，以判斷其總材積是否超出車輛的承載上限。若發現超載，則該路線需進行店鋪抽取，以移除部分店鋪使剩餘配送需求符合車輛容量限制。由於本問題屬於多條路線之決策問題，且具有明確之路線結構，若以單一族群之 GA 進行搜尋，需將所有路線之抽取決策展平成單一染色體表示，容易破壞原有路線結構資訊，並降低搜尋效率與解之品質。因此，本研究採用 CCGA 進行求解，CCGA 透過分治概念將整體抽取問題拆解為多個子問題，每一子問題對應於特定路線之店鋪抽取決策各子族群獨立進行演化搜尋，並透過合作機制整合各子族群之代表解，以構建完整之全域解。經抽取之店鋪將納入候選店鋪集合，作為後續再分配與支援線規劃之基礎，確保主線在不違反材積量限制的情況下，仍能維持配送效率。

#### 3.2.3.2 候選店鋪分配 (Store Assignment)

候選店鋪分配階段主要將候選店鋪集合中的店鋪重新插入至尚有剩餘容量之主線中，以提升既有主線之利用率並降低新增支援線之需求。由於本問題本質上屬於序列式插入決策問題，插入店鋪會影響主線之剩餘容量與路線結構，進而改變後續店鋪之可行插入空間與成本，因此本研究採用 HACO 結合 VND 進行求解。於解建構過程中，透過費洛蒙與啟發式資訊引導店鋪之插入決策，同時考量車輛容量與時間窗限制，其中容量

限制確保各主線總需求不超過車輛承載上限，而時間窗限制則透過 OSRM 計算行駛時間，以驗證配送是否可於允許時間範圍內完成服務。僅有在同時滿足上述限制之條件下，店鋪才會被指派至既有主線；否則將暫時保留於支援線候選集合中。經由多次迭代後，演算法依據目標函數選擇最佳解作為最終之店鋪分配結果。

### 3.2.3.3 支援線規劃 (Support Route Planning)

當候選店鋪無法成功分配到既有主線時，則進入支援線規劃階段，此階段本質上屬於 VRPTW，需同時考量車輛容量限制、時間窗限制與行駛成本等約束條件限制。因此本研究採用 MACS 並結合 VND 進行最佳化。MACS 透過雙蟻群架構，以階層式目標進行搜尋，優先最小化車輛數量，並進一步降低總行駛距離，以提升整體解之品質。在路線建構過程中，結合 VND 進行局部改善，以提升解品質並避免陷入局部最佳解。所有支援線均需滿足總行駛時間限制與時間窗約束，並透過路網服務計算實際行駛時間，以確認各店鋪之到達時間是否落在允許時間區間內，確保路線具備實務可行性。

### 3.2.4 評估模組 (Evaluation Module)

評估模組旨在於比較不同的路線規劃結果，以驗證本研究提出方法之效果。此模組會依據設定的評估指標比較路線規劃結果，而其產出的結果可用來檢視系統的路線規劃表現，並判斷是否符合實務需求。

#### 3.2.4.1 路線評估 (Routes Evaluation)

路線評估主要用於比較物流公司人工編排路線與本研究之系統產生之最佳化結果，以驗證方法的效果與實務可行性。評估指標包含車輛數量、路線行駛距離、路線行駛時間、路線平均裝載率及店鋪準點率。透過分析與比較系統規劃與手動編排結果之間差異。



### 3.2.5 API 模組 (API Module)

此模組負責與外部服務進行串接，以取得實際道路網路之行駛資訊與路線規劃結果。透過該模組，系統能依據規劃的配送順序與店鋪位置查詢實際行駛距離與所需時間，並將查詢結果返回系統，確保與實際規劃一致。若在驗證過程中發現有店鋪無法合理規劃進任何路線，系統會將其重新加入候選店鋪集合，再次執行路線模組的規劃流程。

#### 3.2.5.1 Open Source Routing Machine

OSRM 為基於 OpenStreetMap 之開源路由引擎，能夠計算路網中任意兩點間的最短路徑及預估行車時間。本研究利用 OSRM 建立完整之距離矩陣與時間矩陣，並將其作為路線模組與分配模組之基礎輸入資料。相較於幾何距離或簡化估算方法，OSRM 能夠反映實際道路網路結構、單行道限制與道路層級差異，因此可提升路線規劃之真實性與實務適用性。

#### 3.2.5.2 Routes API

Routes API 為 Google Maps Platform 提供的服務之一，整合原有 Directions API 和 Distance Matrix API 的功能。其功能在於計算兩個或多個地點之間的最佳路徑，並回傳逐步導航指示、距離與預估行駛時間。Routes API 支援多種運輸模式，包括駕車、步行、自行車與大眾運輸，並可以結合即時交通資料估算行駛時間。本研究以 Routes API 作為路線選擇與配送規劃的基礎工具，首先根據實際行駛時間完成店鋪分配，最終再取得完整配送路線，用於後續實際路線呈現與分析。

### 3.2.6 最佳化配送路線資料 (Optimized Route Data)

最佳化配送路線資料為本研究系統經處理後所產生之最終配送結果，其資料結構與原始路線資料相似，但內容已透過最佳化流程進行重新調整與修正。該資料主要記錄經系統規劃後之各車次配送順序及其對應之店鋪配置，以反映實際最佳化後之配送狀態，並作為後續評估分析與實務應用之基礎。

### 3.3 Cooperative Co-evolutionary Genetic Algorithm

本研究採用 CCGA 作為店舖抽取之搜尋機制，主要原因在於本問題具有明顯之可分解特性。由於各主線之店舖抽取決策彼此間呈現低度耦合，不同主線間之解評估主要透過整體車輛需求與路線可行性間接影響，因此適合採用分解策略將問題拆分為多個子群體進行協同優化，並藉由合作評估機制整合各子族群之貢獻。

#### 3.3.1 編碼(Encoding)

店舖抽取問題以二進位向量表示，每個個體對應一主線之店舖抽取結果。各基因對應一間店舖之決策，其中 1 表示店舖被抽取至候選店舖集合，0 則表示維持於原主線配置。式(3.14)為本研究所採用之二進位向量表示方式用以描述單一子族群之店舖抽取解。

$$Z_{k,n}^g = [z_{k,n,1}^g, z_{k,n,2}^g, \dots, z_{k,n,|M_k|}^g] \quad (3.14)$$

#### 3.3.2 選擇(Selection)

本研究於 CCGA 中採用輪盤選擇法(Roulette Wheel Selection)作為父代個體選擇機制，依據個體適應值大小決定其被選取之機率。適應值較高個體具有較高之被選取機率，以確保優良基因得以保留並傳遞至下一代。此外，本研究亦採用精英保留機制(Elitism)，將當前族群中適應值最佳部分個體直接保留至下一世代，以避免優良解在演化過程中遺失。式(3.15)為輪盤選擇法之機率計算方式，用以決定父代個體被選取之機率。

$$p_{k,n}^g = \frac{F_{k,n}^g}{\sum_{r=1}^{N_{sub}} F_{k,r}^g} \quad (3.15)$$

#### 3.3.3 交配(Crossover)

本研究採用均勻交配(Uniform Crossover)作為交配方法。交配僅於同一子族群內之個體間進行，以維持各子族群之結構。依據交配機率判斷是否進行交配，若未達交配條

件，則將父代個體複製至子代以保留既有解結構。當進行交配時，兩個父代個體針對其染色體中各基因位置，以相同機率自兩親代中選取基因，產生子代個體。

### 3.3.4 突變(Mutation)

本研究採用 Bit Flip Mutation 作為突變機制，作用於子族群內之個體。突變依據突變機率隨機選取基因位元進行翻轉，當基因值為 0 時將其轉換為 1，反之則由 1 轉換為 0，以維持族群多樣性並避免演化過程過早收斂。

### 3.3.5 適應值(Fitness)

CCGA 採用合作式評估機制進行適應值計算。由於各子族群僅負責部分決策，因此需結合其他子族群之代表解(context solution)共同形成完整候選解後進行評估。本研究於候選店鋪分配採用貪婪演算法，支援線規劃則採用近鄰演算法進行適應值評估。由於本問題同時涉及最小化車輛使用數量與店鋪抽取數量，因此採用加權線性組合方式進行整合。式(3.16)為適應值函數，其透過支援線數量與總行駛距離之加權組合評估解之品質。

$$F(S) = w_1 * f_{vehicle}(S) + w_2 \cdot |R| + f_{travel}(S) \quad (3.16)$$

### 3.3.6 解的修復或可行性處理

由於交配與突變等基因操作可能破壞原有解之結構，因此於演化過程中導入修復機制，以維持族群解之可行性。本研究設計一套針對店鋪抽取之啟發式修復機制，使所有進入適應值評估之解皆為可行解。該方法採用基於成本效率之啟發式修復策略，將店鋪抽取問題轉換為基於成本評估之修復決策機制，用以決定於修復過程中優先被移出主路線之店鋪。當某一路線之剩餘店鋪集合超過車輛容量限制時，透過成本函數計算各店鋪之抽取權重，以決定被移出主路線之店鋪。式(3.17)表示所定義之店鋪抽取權重函數。

$$w_i = \frac{C}{n} + \theta_i \cdot d_{0,i} \quad (3.17)$$

### 3.4 Hybrid Ant Colony Optimization

本研究採用 HACO 作為候選店鋪分配之搜尋機制，以決定候選店鋪與主路線之對應關係。HACO 結合 ACO 與 VND，其中 ACO 於解建構過程中透過費洛蒙資訊與啟發式資訊逐步完成候選店鋪分配，以進行全域搜尋與候選解生成，並藉由費洛蒙更新機制強化較佳解之搜尋方向，VND 則針對 ACO 所產生之分配結果進行多鄰域區域搜尋，以提升解的開發能力並進一步改善整體分配品質。透過整合 ACO 與 VND 之搜尋機制，本研究方法能在兼顧解品質與搜尋效率之情況下，產生較佳之候選店鋪分配結果。

#### 3.4.1 候選店鋪插入機制

在店鋪選擇與路線構建過程中，螞蟻需決定尚未分配之候選店鋪應插入至哪一條配送路線，以及其於該路線中的插入位置。螞蟻根據費洛蒙濃度、啟發式資訊及目前路線結構狀態等因素，計算各候選店鋪插入至不同路線位置之選取機率。其中，費洛蒙資訊反映歷史搜尋過程中較佳之插入決策經驗，用以引導搜尋方向，啟發式資訊則用於評估當前插入操作對整體配送成本之影響，以提升解構建效率與品質。式(3.18)表示候選店鋪之選擇機率，其值根據費洛蒙濃度、啟發式資訊計算取得。式(3.19)為啟發式資訊，用於反映插入路徑距離及載重成本。式(3.20)表示插入路徑距離之成本，而式(3.21)則用來評估車輛剩餘載容量與店鋪需求量的關係。式(3.22)為啟發式資訊中各項成本權重之限制。

$$p_{i,j,u,k}^a = \frac{\tau_{i,u}^\alpha \cdot \eta_{i,j,u,k}^\beta}{\sum_{z \in N_i^a} \tau_{i,z}^\alpha \cdot \eta_{i,j,z,k}^\beta} \quad (3.18)$$

$$\eta_{i,j,u,k} = \frac{1}{\alpha_1 \cdot C_{i,j,u}^0 + \alpha_2 + C_{i,j,u}^1} \quad (3.19)$$

$$C_{i,j,u}^0 = d_{i,u} + d_{u,j} - d_{i,j} \quad (3.20)$$

$$C_{i,j,u}^1 = Q_k - L_k - v_u \quad (3.21)$$

$$\alpha_1 + \alpha_2 = 1 \quad (3.22)$$

### 3.4.2 費洛蒙初始化與更新

為引導螞蟻於搜尋過程中朝向目標進行探索，本研究以整體目標函數作為評估依據，綜合考量配送支援線總行駛距離，以求得整體規劃效益最佳之路線規劃方案。式(3.23)表示初始費洛蒙濃度之設定方式，以貪婪演算法所產生之初始解作為基準，根據其目標函數值進行初始化。式(3.24)為區域費洛蒙更新公式，用於解構建過程中，當螞蟻每次完成節點移動後即對所經路徑之費洛蒙進行更新，透過降低該路徑之費洛蒙濃度，以減少其被後續螞蟻重複選取之機率，從而提升搜尋過程之多樣性。式(3.25)為全域費洛蒙更新公式，用於每次迭代結束後，依據當前較佳解對其路徑上的費洛蒙進行強化。同時結合費洛蒙揮發機制，避免濃度無限制累積，防止搜尋過早集中於特定區域，進而引導整體搜尋逐步收斂至較佳解。式(3.26)定義費洛蒙之增量計算方式，針對當前迭代中所選取之最佳解所包含路徑進行費洛蒙更新，其增量依據目標函數值決定，使品質較佳之解能在後續搜尋中具有較高被選擇的機率，以提升整體求解品質。

$$\tau_0 = \frac{1}{|R| \cdot F(S_{gr})} \quad (3.23)$$

$$\tau_{i,j} = (1 - \rho) \cdot \tau_{i,j} + \rho \cdot \tau_0 \quad (3.24)$$

$$\tau_{i,j} = (1 - \rho) \cdot \tau_{i,j} + \rho \cdot \Delta\tau_{i,j} \quad (3.25)$$

$$\Delta\tau_{i,j} = \begin{cases} \frac{1}{F(S)} & \text{if the edge } (i,j) \in S, \\ 0 & \text{otherwise,} \end{cases} \quad (3.26)$$

### 3.4.3 Variable Neighborhood Descent

為進一步提升候選店鋪分配之解品質，本研究於所建構之初始解基礎上，結合 VND 進行區域搜尋。透過多種鄰域結構，並依序於不同鄰域中進行解的改進。本研究中採用鄰域結構包含 2-opt、Swap、Relocate 與 Cross Exchange。2-opt 用於對單一路線進行路徑段反轉以縮短行駛距離，Swap 透過交換店鋪之服務順序或所屬路線，重新配置配送結構，Relocate 則用於調整單一店鋪於路線中的位置或跨路線重新分配，而 Cross

Exchange 則透過在兩條不同路線間交換一段連續的店舖序列，以同時調整多個節點之配置，進而提升整體路線之協調性與配送效率。為避免破壞原始主線之既有配送結構，本研究之 VND 僅針對「未分配至其原主線之候選店舖」進行鄰域操作與調整，而原本已位於對應主線之店舖則不參與區域搜尋。藉此可在維持既有主線穩定性的前提下，針對需重新分配之店舖進行局部改善，以降低搜尋空間並提升搜尋效率。透過上述多鄰域操作，VND 可在不同層級上對解進行調整與改善，以兼顧搜尋效率與解品質，提升整體候選店舖分配之結果。式(3.27)表示第  $k$  個鄰域結構對當前解作用後生成的新解集合。

$$S^* = \arg \min_{S' \in N_k(S)} F(S') \quad (3.27)$$

#### 3.4.4 目標函數值

HACO 之目標函數用以衡量候選店舖分配結果之整體配送成本，其計算方式如式(3.28)所示。本階段不再採用貪婪式分配估算，直接針對候選店舖分配結果進行演化與評估。

$$F(S) = w_1 * f_{vehicle}(S) + f_{travel}(S) \quad (3.28)$$



## 3.5 Multiple Ant Colony System

本研究採用 MACS 結合 VND，以解決支援線配送路線規劃問題。在 MACS 架構中，不同蟻群分別針對不同目標進行搜尋，以在降低車輛數量的同時降低總行駛距離。其中，ACS-VEI 以最小化車輛數量為主要目標，ACS-DIST 蟻群以最小化總行駛距離為目標，以降低整體運輸成本。在解構建過程中，MACS 負責全域探索與候選解生成，並透過費洛蒙機制引導搜尋方向，而 VND 則針對既有解進行多鄰域區域搜尋，以強化解的開發能力並進一步降低整體配送成本。透過整合多蟻群與 VND 的區域搜尋機制，本研究方法能在兼顧解品質與計算效率的情況下，產生具實務可行性之配送路線規劃結果。

### 3.5.1 ACS-VEI

ACS-VEI 為 MACS 架構中負責最小化車輛數量之子蟻群，其主要目的在於透過路線構建過程提升客戶覆蓋效率，並減少所需車輛數量。於解構建階段，螞蟻以逐步插入方式生成路線解，並透過記憶陣列機制，引導演算法優先選擇較少被納入解之客戶節點，以提升整體路線填充率。

### 3.5.2 ACS-DIST

ACS-DIST 為 MACS 架構中以最小化總行駛距離為目標之子蟻群，其主要目標是在既定車輛數量限制下最小化總行駛距離。在建構解階段，螞蟻逐步形成完整之配送路線，並於建構完成後結合 VND 進行局部搜尋以強化解品質。於每次迭代結束後，將所有候選解與目前全域最佳解進行比較，並據此更新最佳解及其對應之費洛蒙分佈。透過反覆建構、局部搜尋與費洛蒙更新機制，使搜尋逐步收斂至較佳之最短路徑解。

### 3.5.3 店鋪節點轉移策略

在店鋪選擇與路線構建過程中，螞蟻根據費洛蒙濃度、距離啟發資訊等因素，計算每個尚未拜訪候選店鋪被選取的機率。式(3.29)表示候選店鋪之選擇機率，其值根據費洛



蒙濃度、距離啟發資訊計算取得。式(3.30)為距離啟發資訊，表示當前店鋪到各候選店鋪之距離，用於反映路徑距離及成本。式(3.31)表示當前店鋪至候選店鋪之移動成本。該式於原始距離基礎上加入節點記憶項，以降低長時間未被選取節點之相對距離，使其在構建解時更容易被選擇，進而提升搜尋多樣性。

$$p_{i,j}^a = \frac{\tau_{i,j} \cdot \eta_{i,j}^\beta}{\sum_{z \in N_i^a} \tau_{i,z} \cdot \eta_{i,z}^\beta} \quad (3.29)$$

$$\eta_{i,j} = \frac{1}{\Phi_{i,j}} \quad (3.30)$$

$$\Phi_{i,j} = \max(1.0, d_{i,j} - IN_j) \quad (3.31)$$

### 3.5.4 費洛蒙初始化與更新

本研究於支援線規劃階段之費洛蒙初始化與更新機制，採用與前述 HACO 相同方式進行設計，其初始費洛蒙濃度設定方式如式(3.23)所示。於解建構過程中，螞蟻每完成一次節點移動或店鋪插入後，即依據式(3.24)進行區域費洛蒙更新，以降低已選擇路徑邊之費洛蒙濃度，提升搜尋過程之多樣性；而於每次迭代結束後，則依據當前最佳解，利用式(3.25)進行全域費洛蒙更新，以強化高品質解所對應之路徑邊。此外，費洛蒙增量之計算方式則如式(3.26)所示，其增量大小依據目標函數值決定，使品質較佳之解在後續搜尋中具有較高被選擇之機率，進而提升整體求解品質與收斂能力。

### 3.5.5 Variable Neighborhood Descent

本研究於 ACS-DIST 結合 VND 進行區域搜尋，以進一步提升支援線配送路線之規劃結果。由於 ACS-DIST 在解空間探索過程中，雖具備良好的全域搜尋能力，仍可能因資訊素收斂而陷入局部最佳解，因此透過 VND 對既有解進行改善。VND 流程與鄰域搜尋機制同 HACO 所述，採用 2-opt、Swap、Relocate 與 Cross Exchange 作為鄰域結構，其中第 k 個鄰域結構對當前解作用後生成之新解集合，其表示方式同式(3.27)所示。

### 3.6 Complexity and Problem Decomposition Analysis

在固定主線店鋪配送順序之物流配送規劃中，路線配送由主線與支援線配送組成。此問題結構同時涉及組合選擇與路徑建構兩種不同性質之決策。若不進行問題分解 (decomposition)，則需於單一階段中同時決定店鋪抽取、主線重新分配與支援線路徑規劃，使整體解空間隨店鋪數量增加呈指數級成長，導致搜尋空間過於龐大。此外，單一決策可能同時影響主線容量、支援線需求與配送路徑結構，容易產生大量不可行解，進而降低搜尋效率與解品質。基於上述問題特性，本研究採用問題分解策略，依序將問題劃分為主線店鋪抽取、候選店鋪再分配與支援線規劃三個階段，依據問題特性將不同演算法策略應用於相對應階段，以縮小搜尋過程中的解空間，提升整體搜尋效率與解品質。

首先，主線店鋪抽取問題在固定店鋪配送順序的前提下，需決定哪些店鋪應被抽取以解除超載限制，此類問題可視為一個二元選擇問題。由於實務配送作業中存在固定主線配送之路線規劃，配送順序多已固定，因此在店鋪抽取過程中，路線中店鋪之服務順序保持不變，僅針對店鋪是否保留進行決策。若主線平均包含  $n$  間店鋪，則每條主線之抽取組合為  $2^n$ ，整體解空間約為  $2^{n \cdot m}$ ，隨著店鋪及主線數量增加，解空間將呈指數成長。此外，在抽取階段中，各主線之抽取決策主要受到自身限制影響。基於此，本研究根據既有主線架構進行問題分解，將各主線視為子問題，並於演化過程中分別進行搜尋與最佳化。相較於以單一染色體同時處理所有主線抽取決策之方式，此方法可降低搜尋空間複雜度。因此，本研究採用 CCGA 進行求解，透過其適用於高維組合選擇問題之特性，能在龐大的搜尋空間中進行有效搜尋，同時透過族群演化機制維持解的多樣性，並降低搜尋過程陷入局部最佳解的風險，以尋找符合容量限制且具良好配送效率的抽取結果。

其次，候選店鋪再分配屬於序列式問題，需依序決定每一候選店鋪之插入位置，使各主線同時滿足容量與時間窗限制。若以  $r$  表示候選店鋪數量， $m$  表示主線數量，且假設每一主線平均具有  $n$  間店鋪，則在忽略路線結構動態變化與可行性限制之簡化假設下，其解空間可視為由逐一插入決策所構成之組合空間，近似表示為  $(m \cdot (n + 1))^r$ ，其中  $n+1$

表示可插入於任兩店舖間之位置，隨著候選店舖數量增加，解空間呈指數級成長。因此，本研究採用 HACO 進行求解，透過費洛蒙機制與啟發式資訊引導候選店舖之分配決策，並結合 VND 對既有解進行區域改善，以提升取得可行且高品質解之機率。

完成候選店舖再分配後，支援線規劃採用 MACS 架構，此階段的核心問題可視為 VRPTW，並考量不允許提前到達等待之配送限制。由於在分配每個店舖時需依序決策，先前的路徑安排會直接影響後續店舖與其配送成本，因此屬於具有高度路徑依賴性之序列式決策問題。VRPTW 本身為典型之 NP-hard 組合最佳化問題，其解空間包含所有可能之客戶拜訪順序與路線組合，若僅考慮單一路線之客戶拜訪順序，其解空間上界為  $n!$ ，並隨問題規模增加呈階乘級成長。因此採用 MACS 進行路徑建構與解空間探索，再透過 VND 對既有解進行區域改善，以提升解品質，同時考量店舖間距離、配送時間窗與車輛狀態更新。此搜尋機制能在兼顧搜尋效率與解穩定性的同時，產生具高度實務可行性的支援線規劃方案。此外，本研究採用階層式目標(Hierarchical objective)進行最佳化，首先以最小化配送車輛數量為優先目標，再進一步最小化總行駛距離。此種分階段之決策方式可反映實務營運中對不同目標之優先順序與偏好。

綜上所述，本研究將原始整體配送問題由單一組合最佳化問題，轉化為具層次結構之分解最佳化框架。透過將問題拆解為具不同決策特性之子問題，使原本呈指數級成長之解空間被分解至較低維度之子空間中，進而降低整體搜尋複雜度，使演算法能在合理的計算時間內，於複雜且具實務限制的配送環境中，產生高品質且可執行的配送結果。

表 1 演算法對應關係表

| 子問題                    | 求解演算法 |
|------------------------|-------|
| Store Extraction       | CCGA  |
| Store Assignment       | HACO  |
| Support Route Planning | MACS  |

## 第四章 實驗與討論

本章針對前章所提出的方法進行實驗設計與結果分析。內容包括實驗環境、研究方法的實作流程及實驗數據彙整，並對實驗結果進行說明、分析與討論。

### 4.1 實驗環境

本研究進行實驗之硬體設備規格如表 2 所示。

表 2 實驗設備規格表

|            |                                |
|------------|--------------------------------|
| 作業系統(OS)   | Windows 11                     |
| 中央處理器(CPU) | Intel® Core™ i7-13700 5.20 GHz |
| 顯示卡(GPU)   | Intel® UHD graphics 770        |
| 記憶體        | 24GB                           |
| 硬碟         | 1TB SSD                        |

本研究採用 Python 程式語言進行系統之設計與開發，整合歷史配送資料與地理資訊，模擬實際物流配送情境，建置一套可支援物流路線編排之規劃系統，以提供實務上可行的配送路線參考，並以此作為演算法驗證與比較之基礎。

### 4.2 實驗資料集

本研究之實驗資料集使用物流公司提供之人工編排路線資料，其內容與結構如圖 2 所示。內容涵蓋配送車次、店鋪名稱、表定與預定到店時間、店鋪所需材積量、車輛總材積量及裝載率。配送車次包含車輛編號與店鋪編號，車輛編號又可區分為主線車輛編號與支援線車輛編號，主線車輛編號之格式為一數字搭配一字母，支援線車輛編號之格式則為三個數字。店鋪編號則由主線車輛編號加上物流公司初步訂定之店鋪服務順序所組成。表定到店時間定義店鋪的時間窗限制，其可接受之表定到店時間為前一小時至後三十分鐘。

| DC: 林口DC          |       |        |                     |                     |      |          |
|-------------------|-------|--------|---------------------|---------------------|------|----------|
| 配別名稱: 常溫          |       |        |                     |                     |      |          |
| 進貨指定日: 2022-12-07 |       |        |                     |                     |      |          |
| 不可大車              | 車次    | 店名     | 表定時間                | 預定時間                | 貨量   | 裝載率      |
|                   | 101   | 林口DC   |                     |                     | 5.5  | 0.763889 |
| False             | 3I01  | 林口新僑大店 | 2022-12-06 17:00:00 | 2022-12-06 17:00:00 | 0.56 |          |
| False             | 3I02  | 林口工六店  | 2022-12-06 17:20:00 | 2022-12-06 17:16:00 | 0.41 |          |
| False             | 3M01  | 新莊海山店  | 2022-12-06 18:20:00 | 2022-12-06 17:43:00 | 1.76 |          |
| False             | 4I05  | 新莊樹成店  | 2022-12-06 18:50:00 | 2022-12-06 18:05:00 | 0.72 |          |
| False             | 4I06  | 新莊福前店  | 2022-12-06 19:10:00 | 2022-12-06 18:21:00 | 1.09 | 0.681944 |
| False             | 4I07  | 新莊自強店  | 2022-12-06 19:25:00 | 2022-12-06 18:35:00 | 0.96 |          |
| False             | 101DC | 林口DC   | 2022-12-06 19:05:00 | 2022-12-06 20:03:40 | 0    |          |
|                   | 102   | 林口DC   |                     |                     | 4.91 |          |
| False             | 4H02  | 林口新高中店 | 2022-12-06 17:15:00 | 2022-12-06 20:52:44 | 1    |          |
| False             | 4H03  | 林口國宅店  | 2022-12-06 17:30:00 | 2022-12-06 21:09:00 | 1.01 | 0.75     |
| False             | 3D05  | 三合店    | 2022-12-06 18:45:00 | 2022-12-06 17:45:06 | 0.84 |          |
| False             | 3D07  | 中央南路店  | 2022-12-06 19:15:00 | 2022-12-06 18:03:45 | 0.88 |          |
| False             | 3D08  | 東陽店    | 2022-12-06 19:30:00 | 2022-12-06 18:35:25 | 1.18 |          |
| False             | 102DC | 林口DC   | 2022-12-06 19:54:00 | 2022-12-06 23:31:00 | 0    |          |
|                   | 103   | 林口DC   |                     |                     | 5.4  | 0.75     |
| False             | 5G01  | 新莊盛華店  | 2022-12-06 22:15:00 | 2022-12-06 22:31:31 | 1.39 |          |
| False             | 5G03  | 新莊新建明店 | 2022-12-06 22:45:00 | 2022-12-06 22:52:31 | 0.96 |          |
| False             | 5A07  | 三重大智店  | 2022-12-06 23:45:00 | 2022-12-06 23:23:31 | 1.02 |          |
| False             | 5C07  | 三重新正店  | 2022-12-07 00:30:00 | 2022-12-06 23:41:01 | 1.12 |          |
| False             | 5B08  | 三重致學店  | 2022-12-07 00:45:00 | 2022-12-06 23:54:31 | 0.91 | 0        |
| False             | 103DC | 林口DC   | 2022-12-07 00:03:00 | 2022-12-07 01:09:01 | 0    |          |

圖 2 物流公司人工編排路線資料

由於初始取得資料為人物物流公司人工編排路線資料，為進行後續最佳化運算，本研究首先進行資料預處理，再執行後續路線最佳化。實驗資料集涵蓋 26 個作業日之人工編排路線資料，各作業日之主線數量介於 81 至 88 條之間，多數案例約為 81 條主線，如圖 3 所示。每條路線涵蓋之店鋪數量約介於 3 至 15 間，每日配送之店鋪總數約為 717 至 769 間，其分布情形如圖 4 所示。每日配送店鋪組成、位置以及各店鋪之需求材積量皆隨訂單需求而變動。圖 5 顯示各作業日之路線載重率分布。

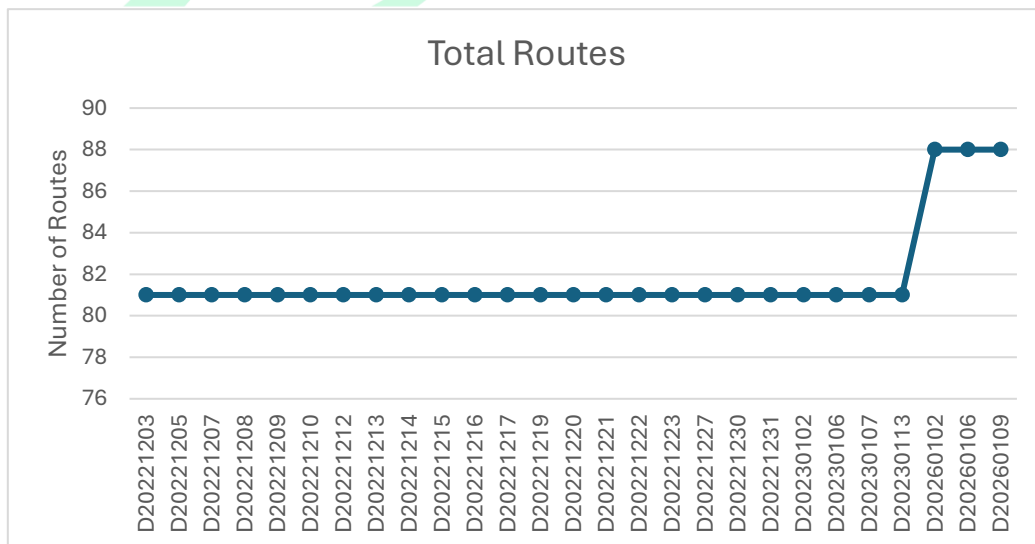


圖 3 原始路線各實例之主線車輛數量



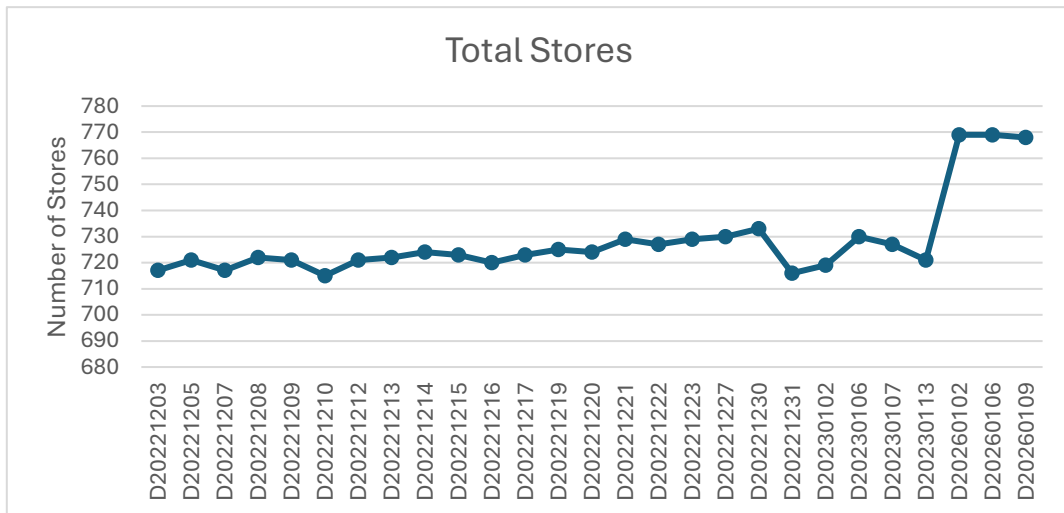


圖 4 原始路線各實例之店鋪數量

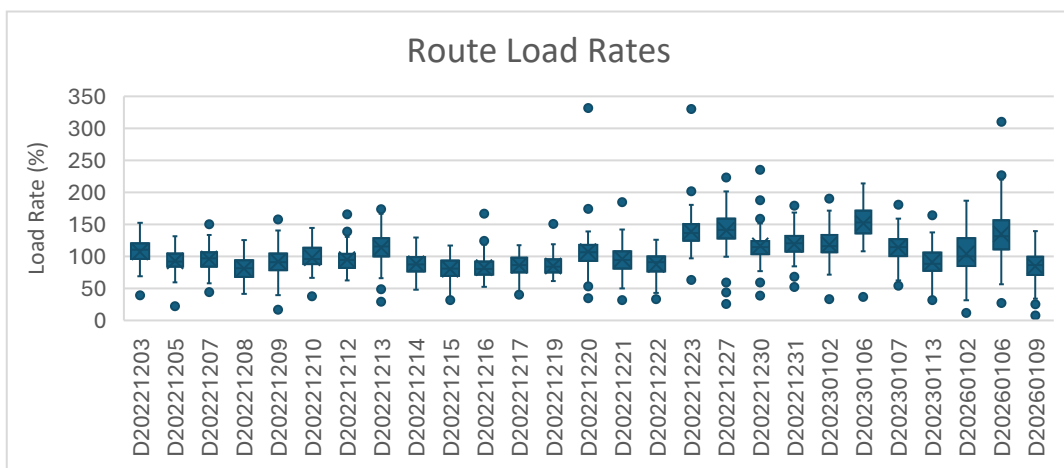


圖 5 原始路線各實例車輛裝載率分佈

### 4.3 評估基準

本研究以物流公司所提供手動規劃數據作為比較基準，針對總路線數量、總行駛距離、總行駛時長、路線平均裝載率、準點率及規劃所需時間與物流公司進行比較。總路線數量為完成當日配送所需的車輛總數，數量越少代表固定成本越低，定義為效果較佳。總行駛距離則對應至變動成本，在車輛數量相同時，數值越低代表路徑規劃越具效率，定義為效果較佳。平均路線裝載率代表容量限制下的配送資源利用效率，數值越高定義為效果較佳。準點率則反映配送服務品質，表示店鋪能於指定時間內完成配送，數值越高定義為效果較佳。規劃所需時間越短定義為效果較佳。

## 4.4 超參數設定與校準

本研究中各階段所使用的演算法針對不同問題，其超參數設定對求解品質與收斂速度均具有重要影響。因此，本研究針對各階段所使用的演算法進行參數校準，以確保每個演算法在其所解決的問題上達到最佳效能。

### 4.4.1 CCGA 之超參數設定

本研究將 CCGA 應用於主線店舖抽取問題，並針對迭代次數、子族群大小、交配機率與突變機率等主要超參數進行設定與調整。由於 CCGA 採用協同演化架構，各子族群需透過合作式評估共同形成完整解，因此超參數設定將影響搜尋效率、解品質與子族群間之協調效果。其中，迭代次數影響演算法之搜尋深度與收斂程度，子族群大小影響每代參與演化之解數量，交配機率影響不同群組結構間之基因重組能力，突變機率則影響族群多樣性與跳脫局部最佳解之能力。CCGA 之超參數設定如表 3 所示。

表 3 CCGA 超參數設定

| 問題類型             | 超參數                     | 設定值 |
|------------------|-------------------------|-----|
| Store Extraction | Generation              | 200 |
|                  | Subpopulation size      | 20  |
|                  | Crossover rate          | 0.8 |
|                  | Mutation rate           | 0.1 |
|                  | Early stopping patience | 40  |

### 4.4.2 HACO 之超參數設定

本研究將 HACO 應用於候選店舖分配問題，並針對迭代次數、螞蟻數量、費洛蒙蒸發速率與啟發函數重要度等超參數進行設定與調整。由於 HACO 透過費洛蒙機制與啟



發式資訊共同引導解建構過程，因此上述參數將直接影響搜尋過程收斂速度與最終解品質，HACO 之超參數設定如表 4 所示。

表 4 HACO 超參數設定

| 問題類型             | 超參數                     | 設定值 |
|------------------|-------------------------|-----|
| Store Assignment | Iteration               | 200 |
|                  | Number of Ants          | 50  |
|                  | Evaporation rate        | 0.7 |
|                  | Heuristic factor        | 1   |
|                  | Early stopping patience | 40  |

#### 4.4.3 MACS 之超參數設定

本研究將 MACS 應用於 VRPTW，其搜尋行為主要受搜尋時間、螞蟻數量、費洛蒙蒸發速率與啟發函數重要度四個超參數影響。費洛蒙蒸發速率決定費洛蒙隨時間消散的速度，值越大演算法對新資訊反應越靈敏，有助於探索新解，而值過小則傾向強化已發現的良好路徑，加速收斂。啟發函數重要度則表示螞蟻對啟發式資訊的重視程度，值較大可幫助快速找到可行解，但過高會抑制費洛蒙正回饋機制的作用。MACS 超參數之設定如表 5 所示。

表 5 蟻群演算法超參數設定

| 問題類型                   | 超參數              | 設定值 |
|------------------------|------------------|-----|
| Support Route Planning | Time limit       | 100 |
|                        | Number of Ants   | 50  |
|                        | Evaporation rate | 0.5 |
|                        | Heuristic factor | 7   |

## 4.5 自動化路線編排流程實作

本節旨在驗證本研究所提出之自動化路線編排流程，相較於人工編排方式所能達成之整體規劃成效。透過實際執行自動化編排流程，評估其於各項路線規劃指標上的表現，藉此檢視該流程是否具備改善人工編排結果之潛力。

本節將依據第三章所提出之方法與演算法設計，說明自動化路線編排流程於實作層面之執行方式，並依循實際編排順序，介紹各模組於整體流程中實作細節，以利後續實驗結果之分析與討論。

### 4.5.1 資料前處理

本節內容對應第三章所述之資料前處理，主要針對原始取得相關資料進行彙整、轉換與清理，以建立後續自動化路線編排流程所需之結構化資料集。相關資料包含路網資料、各店滯店時間、店舖主檔及既有人工編排配送路線資料。

在資料前處理過程中，首先將資料來源進行格式統一與欄位對應，並依據路線編號與店舖編號進行資料整合，以建立每條路線對應之配送路線資料與店舖清單。同時，針對缺失值、不合理數值及重複紀錄進行檢查與修正，以確保資料的一致性與正確性。此外，亦將時間欄位轉換為統一格式，並整理各店舖之地理座標、配送時間窗、店舖所需材積量與滯店時間等資訊，作為後續路線規劃與演算法計算之依據。具體範例與說明如圖 6 所示。

|    |                                     |             |
|----|-------------------------------------|-------------|
| 01 | "2M": {                             | //路線 2M     |
| 02 | "dc": {                             |             |
| 03 | "route_id": "2M",                   | //路線 ID     |
| 04 | "total_volume": 2.8099999999999996, | //路線總材積量    |
| 05 | "load_rate": 0.3902777777777777,    | //路線裝載率     |
| 06 | "max_capacity": 7.2,                | //車輛最大材積量上限 |
| 07 | "distance": 30.769,                 | //路線總距離，公里  |
| 08 | "duration": 5795.0                  | //路線總時長，秒數  |
| 09 | },                                  |             |
| 10 | "stores": [                         | //路線配送店舖清單  |
| 11 | {                                   |             |

|    |                                         |            |
|----|-----------------------------------------|------------|
| 12 | "route_id": "2M",                       | //路線 ID    |
| 13 | "store_code": "2M02",                   | //店鋪編號     |
| 14 | "store_id": "1001018520",               | //店鋪 ID    |
| 15 | "store_name": "高鐵二店",                   | //店鋪名稱     |
| 16 | "longitude": 121.515932256166,          | //店鋪位置經度   |
| 17 | "latitude": 25.0481762099715,           | //店鋪位置緯度   |
| 18 | "sched_time": "2022-12-02T18:30:00",    | //店鋪規劃抵達時間 |
| 19 | "earliest_time": "2022-12-02T17:30:00", | //店鋪最早服務時間 |
| 20 | "latest_time": "2022-12-02T19:00:00",   | //店鋪最晚服務時間 |
| 21 | "pred_time": "2022-12-02T18:30:00",     | //店鋪預計抵達時間 |
| 22 | "dwell_time": 733.0,                    | //滯店時間     |
| 23 | "volume": 1.31                          | //店鋪材積量    |
| 24 | },                                      |            |
| 25 | {                                       |            |
| 26 | "route_id": "2M",                       |            |
| 27 | "store_code": "2M03",                   |            |
| 28 | "store_id": "1001018687",               |            |
| 29 | "store_name": "台鐵一店",                   |            |
| 30 | "longitude": 121.516342621662,          |            |
| 31 | "latitude": 25.0481697237322,           |            |
| 32 | "sched_time": "2022-12-02T19:00:00",    |            |
| 33 | "earliest_time": "2022-12-02T18:00:00", |            |
| 34 | "latest_time": "2022-12-02T19:30:00",   |            |
| 35 | "pred_time": "2022-12-02T19:00:00",     |            |
| 36 | "dwell_time": 733.0,                    |            |
| 37 | "volume": 0.95                          |            |
| 38 | },                                      |            |
| 39 | {                                       |            |
| 40 | "route_id": "2M",                       |            |
| 41 | "store_code": "2M04",                   |            |
| 42 | "store_id": "1001021597",               |            |
| 43 | "store_name": "機場捷運店",                  |            |
| 44 | "longitude": 121.515029810015,          |            |
| 45 | "latitude": 25.0485284602299,           |            |
| 46 | "sched_time": "2022-12-02T19:30:00",    |            |
| 47 | "earliest_time": "2022-12-02T18:30:00", |            |
| 48 | "latest_time": "2022-12-02T20:00:00",   |            |
| 49 | "pred_time": "2022-12-02T19:30:00",     |            |
| 50 | "dwell_time": 610,                      |            |
| 51 | "volume": 0.55                          |            |
| 52 | }                                       |            |
| 53 | ]                                       |            |
| 54 | }                                       |            |

圖 6 路線資料格式範例

## 4.5.2 超載路線店舖抽取

在資料前處理完成後，本研究進入店舖抽取階段，針對物流公司原始規劃中因配送需求波動而超載的路線進行調整，以解決路線超載問題。系統首先進行初始族群生成，每個個體代表一組從超載路線中抽取的店舖集合，以構成可行之候選解。

| Algorithm 1 Initial Subpopulation Generation |                                                     |
|----------------------------------------------|-----------------------------------------------------|
| 01                                           | <b>Begin</b>                                        |
| 02                                           | Initialize subpopulation $P_k^0 = \emptyset$        |
| 03                                           | <b>For</b> $n = 1$ to $N_{sub}$ <b>Do</b>           |
| 04                                           | Initialize individual $Z_{k,n}^0$                   |
| 05                                           | <b>While</b> $r_k$ is overloaded <b>Do</b>          |
| 06                                           | Select an unextracted store $i$                     |
| 07                                           | Set gene $z_{k,n,i}^0 = 1$                          |
| 08                                           | <b>End While</b>                                    |
| 09                                           | Add Individual $Z_{k,n}^0$ to subpopulation $P_k^0$ |
| 10                                           | <b>End For</b>                                      |
| 11                                           | <b>End</b>                                          |

圖 7 店舖抽取之初始子族群生成偽程式碼

完成初始族群生成後，系統進入 CCGA 演化階段。在此階段中，演化過程依序執行選擇、交叉與突變操作，每個個體的適應值以整體配送路線之總距離與車輛使用成本衡量，藉此引導演算法尋找最佳解。完成迭代後，演算法輸出最佳個體對應的路線資料及被抽取的店舖清單，作為後續路線重配置與最佳化的基礎，使配送路線能在合理載運限制下運作。基因演算法之偽程式碼如圖 8 所示

| Algorithm 2 Cooperative Co-evolutionary Genetic Algorithm |                                                                         |
|-----------------------------------------------------------|-------------------------------------------------------------------------|
| 01                                                        | <b>Begin</b>                                                            |
| 02                                                        | Initialize context solution $b = \emptyset$                             |
| 03                                                        | <b>For</b> each route $r_k$ in $M$ <b>Do</b>                            |
| 04                                                        | Initialize subpopulation $P_k^0$ //Algorithm 1                          |
| 05                                                        | Evaluate the fitness of each individual in $P_k^0$ combined with $b$    |
| 06                                                        | Update $b$ based on the best individual in $P_k^0$                      |
| 07                                                        | <b>End For</b>                                                          |
| 08                                                        | <b>For</b> each generation $g = 0$ to $G$ <b>Do</b>                     |
| 09                                                        | <b>For</b> each route $r_k$ in $M$ <b>Do</b>                            |
| 10                                                        | Preserve the top $E$ individuals from $P_k^g$ and retain to $P_k^{g+1}$ |
| 11                                                        | <b>While</b> $ P_k^{g+1}  < N_{sub}$ <b>Do</b>                          |
| 12                                                        | $n =  P_k^{g+1}  + 1$                                                   |
| 13                                                        | Select parents $Z_{k,a}^g, Z_{k,b}^g$ from $P_k^g$ based on fitness     |

|    |                                                                                  |
|----|----------------------------------------------------------------------------------|
| 14 | Perform Crossover to produce offspring $Z_{k,n}^{g+1}, Z_{k,n+1}^{g+1}$          |
| 15 | Mutate offspring $Z_{k,n}^{g+1}, Z_{k,n+1}^{g+1}$                                |
| 16 | Add offspring $Z_{k,n}^{g+1}, Z_{k,n+1}^{g+1}$ to the new population $P_k^{g+1}$ |
| 17 | <b>End While</b>                                                                 |
| 18 | Evaluate the fitness of each individual in $P_k^{g+1}$ combined with $b$         |
| 19 | Update $b$ based on the best individual in $P_k^{g+1}$                           |
| 20 | <b>End For</b>                                                                   |
| 21 | <b>End For</b>                                                                   |
| 22 | <b>End</b>                                                                       |

圖 8 店鋪抽取之 CCGA 偽程式碼

### 4.5.3 候選店鋪分配

本階段針對前述程序所產生之候選店鋪，依據其配送材積量與時間窗限制，重新分配至尚具剩餘容量的主線路線中，以最大化主線利用率並減少支援線需求。分配過程中，需同時檢查材積量是否仍符合車輛容量限制，以及時間窗等相關約束條件。

本階段採用 HACO 進行候選店鋪分配。如圖 9 所示，首先透過貪婪策略產生初始解以初始化費洛蒙，並於每次迭代中建構多組螞蟻解。每組解皆代表一組完整的店鋪分配結果，並於建構過程中檢查車輛容量與時間窗限制，以確保解之可行性。此外，於每次迭代完成後，針對當前迭代最佳解導入 VND 局部搜尋機制，以進一步提升整體解品質。

| Algorithm 3 Hybrid Ant Colony Optimization |                                                                  |
|--------------------------------------------|------------------------------------------------------------------|
| 01                                         | <b>Begin</b>                                                     |
| 02                                         | Generate and evaluate solution $S_{gr}$                          |
| 03                                         | Initialize pheromone $\tau_0$                                    |
| 04                                         | <b>While</b> stopping criterion is not met <b>Do</b>             |
| 05                                         | Initialize solution $S_{iter} = \emptyset$                       |
| 06                                         | <b>For</b> each ant $m$ <b>Do</b>                                |
| 07                                         | $S_m$ = Solution Construction //Algorithm 4                      |
| 08                                         | <b>If</b> $\text{cost}(S_m) < \text{cost}(S_{iter})$ <b>Then</b> |
| 09                                         | $S_{iter} = S_m$                                                 |
| 10                                         | <b>End If</b>                                                    |
| 11                                         | <b>End For</b>                                                   |
| 12                                         | Apply VND to improve the solution $S_{iter}$ //Algorithm 5       |
| 13                                         | Update global pheromone $\tau_{i,j}$ based on $S_{best}$         |
| 14                                         | <b>End While</b>                                                 |
| 15                                         | <b>End</b>                                                       |

圖 9 HACO 之偽程式碼

在解的建構階段，首先將所有主路線進行隨機排序，並依序對每條路線進行店舖分配。在每條路線中，針對尚未分配之店舖集合，評估其在不同插入位置之可行性與吸引程度，並結合費洛蒙資訊與啟發式函數計算插入機率，以決定店舖與插入位置。每次成功插入後更新路線之材積量與行駛時間，當當前路線無法再插入任何可行店舖時則停止該路線之更新並切換至下一條路線，若仍有未分配店舖則納入支援集合處理。

| Algorithm 4 Solution Construction |                                                                |
|-----------------------------------|----------------------------------------------------------------|
| 01                                | <b>Begin</b>                                                   |
| 02                                | Initialize a random permutation of all routes                  |
| 03                                | <b>For</b> each route $r_k$ in the permutation <b>Do</b>       |
| 04                                | <b>While</b> $\exists u \in R$ is feasible to insert <b>Do</b> |
| 05                                | Compute $p_{i,j,u,k}^a$ based on $N_i^a$                       |
| 06                                | Select and insert store $u$ into edge $(i,j)$ of $r_k$         |
| 07                                | Update current load $L_k$                                      |
| 08                                | Update time $T_k$                                              |
| 09                                | Update local pheromone $\tau_{i,j}$ based on $\tau_0$          |
| 10                                | Remove $u$ from $R$                                            |
| 11                                | <b>End While</b>                                               |
| 12                                | <b>End For</b>                                                 |
| 13                                | <b>End</b>                                                     |

圖 10 HACO 路線建構之偽程式碼

為提升解品質，本階段進一步採用 VND 作為局部改善策略，以降低配送成本並改善路線結構。於路線內改善階段，僅針對不屬於該主線配置之店舖進行調整，以維持原有主線結構。若於任一鄰域操作中發現可改善目標函數之解，則更新目前解並重新由第一個鄰域開始搜尋，當所有鄰域操作均無法進一步改善目標函數值時，則終止區域搜尋機制。透過上述強化機制，可在兼顧解之可行性之前提下，有效提升整體解品質並降低總配送成本。

| Algorithm 5 Variable Neighborhood Descent |                                                      |
|-------------------------------------------|------------------------------------------------------|
| 01                                        | <b>Begin</b>                                         |
| 02                                        | Initialize solution $S^* = S_m$                      |
| 03                                        | Initialize neighborhood $n = 1$                      |
| 04                                        | <b>While</b> $n \leq 4$ <b>Do</b>                    |
| 05                                        | Initialize best_improvement = 0                      |
| 06                                        | <b>If</b> $n = 1$ <b>Then</b>                        |
| 07                                        | <b>For</b> each route $r_k$ in $S^*$ <b>Do</b>       |
| 08                                        | Apply 2-opt operator to $r_k$ to obtain $S'$         |
| 09                                        | <b>If</b> improvement > best_improvement <b>Then</b> |

|    |                                                              |
|----|--------------------------------------------------------------|
| 10 | Update solution $S^*$ with $S'$                              |
| 11 | <b>End For</b>                                               |
| 12 | <b>Else If</b> $n = 2$ <b>Then</b>                           |
| 13 | <b>For</b> each pair of routes $r_i, r_j$ in $S^*$ <b>Do</b> |
| 14 | Apply relocate operator to $r_i, r_j$ to obtain $S'$         |
| 15 | <b>If</b> improvement > best_improvement <b>Then</b>         |
| 16 | Update solution $S^*$ with $S'$                              |
| 17 | <b>End For</b>                                               |
| 18 | <b>Else If</b> $n = 3$ <b>Then</b>                           |
| 19 | <b>For</b> each pair of routes $r_i, r_j$ in $S^*$ <b>Do</b> |
| 20 | Apply swap operator to $r_i, r_j$ to obtain $S'$             |
| 21 | <b>If</b> improvement > best_improvement <b>Then</b>         |
| 22 | Update solution $S^*$ with $S'$                              |
| 23 | <b>End For</b>                                               |
| 24 | <b>End If</b>                                                |
| 25 | <b>Else If</b> $n = 4$ <b>Then</b>                           |
| 26 | <b>For</b> each pair of routes $r_i, r_j$ in $S^*$ <b>Do</b> |
| 27 | Apply cross exchange operator to $r_i, r_j$ to obtain $S'$   |
| 28 | <b>If</b> improvement > best_improvement <b>Then</b>         |
| 29 | Update solution $S^*$ with $S'$                              |
| 30 | <b>End For</b>                                               |
| 31 | <b>End If</b>                                                |
| 32 | <b>If</b> best_improvement > 0 <b>Then</b>                   |
| 33 | Set $n = 1$                                                  |
| 34 | <b>Else</b>                                                  |
| 35 | Set $n = n + 1$                                              |
| 36 | <b>End If</b>                                                |
| 37 | <b>End While</b>                                             |
| 38 | <b>End</b>                                                   |

圖 11 可變鄰域下降法之偽程式碼

#### 4.5.4 支援線規劃

在完成候選店鋪分配後，本研究進入支援線規劃階段。由於物流公司內部車輛數量有限，當主線車輛於容量與時間限制下仍無法完全承載所有配送需求時，必須將部分剩餘店鋪交由外部車輛進行配送。因此，本階段之目標為針對無法重新分配回主線之店鋪，建立額外支援配送路線，以確保整體配送任務得以完成。

為解決此問題，本研究採用 MACS 進行路線建構與最佳化。透過多個不同搜尋目標之蟻群協同運作，在解空間中同時進行探索與開發。其中，ACS-VEI 子蟻群專注於最小



化支援車輛數量，ACS-DIST 子蟻群則在既定車輛數量下進一步縮短總行駛距離，以提升整體配送效率。MACS 完整偽程式碼，如圖 12 所示。

| Algorithm 6 Multiple Ant Colony System |                                                                          |
|----------------------------------------|--------------------------------------------------------------------------|
| 01                                     | <b>Begin</b>                                                             |
| 02                                     | Generate and evaluate solution $S_{nn}$ using nearest neighbor heuristic |
| 03                                     | Initialize pheromone $\tau$ based on solution $S_{nn}$                   |
| 04                                     | Initialize $S_{best} = S_{nn}$                                           |
| 05                                     | <b>While</b> stopping criterion is not met <b>Do</b>                     |
| 06                                     | $k =  S_{best} $                                                         |
| 07                                     | <b>Activate</b> ACS-VEI( $k - 1$ )                                       |
| 08                                     | <b>Activate</b> ACS-DIST( $k$ )                                          |
| 09                                     | <b>While</b> ACS-VEI and ACS-DIST are active <b>Do</b>                   |
| 10                                     | <b>Wait</b> an improved solution $S_{opt}$ from ACS-VEI or ACS-DIST      |
| 11                                     | $S_{best} = S_{opt}$                                                     |
| 12                                     | <b>If</b> $ S_{best}  < k$ <b>Then</b>                                   |
| 13                                     | <b>Kill</b> ACS-VEI and ACS-DIST                                         |
| 14                                     | <b>End If</b>                                                            |
| 15                                     | <b>End While</b>                                                         |
| 16                                     | <b>End While</b>                                                         |
| 17                                     | <b>End</b>                                                               |

圖 12 MACS 之偽程式碼

ACS-VEI 為 MACS 架構中負責最小化車輛數量之子蟻群，主要目的在於透過路線構建過程提升客戶覆蓋效率，並以此減少所需車輛數量。解構建階段，螞蟻以逐步插入方式生成路線解，並透過記憶陣列機制，引導演算法優先選擇較少被納入解之客戶節點。於每次解建構完成後，將所有候選解與目前全域最佳解進行比較，並依據車輛使用數量作為主要評估指標更新最佳解及其對應之費洛蒙分佈。

| Algorithm 7 ACS-VEI |                                                                                   |
|---------------------|-----------------------------------------------------------------------------------|
| 01                  | <b>Begin</b>                                                                      |
| 02                  | Initialize pheromone                                                              |
| 03                  | Initialize solution $S_{VEI} = S_{nn}$                                            |
| 04                  | <b>While</b> stopping criterion is not met <b>Do</b>                              |
| 05                  | <b>For</b> each ant $m$ <b>Do</b>                                                 |
| 06                  | $S_m$ = Solution Construction //Algorithm 8                                       |
| 07                  | <b>For</b> each customer $j \notin S_m$ <b>Then</b>                               |
| 08                  | $IN_j = IN_j + 1$                                                                 |
| 09                  | <b>End For</b>                                                                    |
| 10                  | <b>End For</b>                                                                    |
| 11                  | <b>If</b> visited_customers( $S_m$ ) > visited_customers( $S_{VEI}$ ) <b>Then</b> |
| 12                  | $S_{VEI} = S_m$                                                                   |
| 13                  | Reset all $IN_j = 0$                                                              |

|    |                                                          |
|----|----------------------------------------------------------|
| 14 | <b>If</b> $S_{VEI}$ <b>is feasible</b> <b>Then</b>       |
| 15 | Return $S_{VEI}$ to MACS                                 |
| 16 | <b>End If</b>                                            |
| 17 | <b>End If</b>                                            |
| 18 | Update global pheromone $\tau_{i,j}$ based on $S_{VEI}$  |
| 19 | Update global pheromone $\tau_{i,j}$ based on $S_{best}$ |
| 20 | <b>End While</b>                                         |
| 21 | <b>End</b>                                               |

圖 13 ACS-VEI 之偽程式碼

相較於 ACS-VEI 著重於車輛數量之最小化，ACS-DIST 為 MACS 架構中負責最小化總行駛距離之子蟻群。為提升解品質，每隻螞蟻於完成路線建構後，再選取其中總行駛距離最佳之解作為區域搜尋對象，並透過 VND 進行改善，使候選解在評估前即具備較佳品質。於每次迭代結束後，根據總行駛距離更新全域最佳解與費洛蒙分佈，引導搜尋朝向較佳路徑收斂。

| Algorithm 8 ACS-DIST |                                                               |
|----------------------|---------------------------------------------------------------|
| 01                   | <b>Begin</b>                                                  |
| 02                   | Initialize pheromone                                          |
| 03                   | <b>While</b> stopping criterion is not met <b>Do</b>          |
| 04                   | Initialize solution $S_{iter}$                                |
| 05                   | <b>For</b> each ant m <b>Do</b>                               |
| 06                   | $S_m$ = Solution Construction //Algorithm 9                   |
| 07                   | <b>If</b> cost( $S_m$ ) < cost( $S_{iter}$ ) <b>Then</b>      |
| 08                   | $S_{iter} = S_m$                                              |
| 09                   | <b>End If</b>                                                 |
| 10                   | <b>End For</b>                                                |
| 11                   | Apply VND to improve the solution $S_{iter}$ //Algorithm 5    |
| 12                   | <b>If</b> $S_{iter}$ is feasible <b>Then</b>                  |
| 13                   | <b>If</b> cost( $S_{iter}$ ) < cost( $S_{best}$ ) <b>Then</b> |
| 14                   | Return $S_{iter}$ to MACS                                     |
| 15                   | <b>End If</b>                                                 |
| 16                   | <b>End If</b>                                                 |
| 17                   | Update global pheromone $\tau_{i,j}$ based on $S_{best}$      |
| 18                   | <b>End While</b>                                              |
| 19                   | <b>End</b>                                                    |

圖 14 ACS-DIST 之偽程式碼

於路線建構過程中，依據物流公司規範及配送需求，逐步生成可行的配送路線。路線以物流中心為起點，並依據物流公司跨區配送原則，將店舖依方位劃分為東、南、西、北四區，並依距離劃分為近區、中區及遠區，據此篩選可行的配送店舖，以避免路線出

現折返情況。完成候選節點篩選後，螞蟻依據費洛蒙資訊及距離等啟發式資訊選擇下一個配送店鋪。每次加入店鋪的過程中，同時檢查車輛是否超過最大容量、抵達時間是否滿足店鋪時間窗限制，以及整體路線行駛時間是否超過最大允許時長。若條件不滿足，則終止當前路線建構，並啟動新路線以分配剩餘店鋪。

| Algorithm 9 Solution Construction |                                                             |
|-----------------------------------|-------------------------------------------------------------|
| 01                                | <b>Begin</b>                                                |
| 02                                | Initialize solution $S_m$                                   |
| 03                                | <b>While</b> $R$ is not empty <b>Do</b>                     |
| 04                                | Initialize a new route $r_k$ for vehicle $k$                |
| 05                                | Initialize $L_k = 0$                                        |
| 06                                | Initialize $T_k = 0$                                        |
| 07                                | <b>While</b> $\exists j \in R$ is feasible to add <b>Do</b> |
| 08                                | Compute $p_{i,j}^a$ based on $N_i^a$                        |
| 09                                | Select next store $j$                                       |
| 10                                | Update current load $L_k$                                   |
| 11                                | Update time $T_k$                                           |
| 12                                | Update local pheromone $\tau_{ij}$ based on $\tau_0$        |
| 13                                | Remove $j$ from $R$                                         |
| 14                                | <b>End While</b>                                            |
| 15                                | Add $r_k$ to $S_m$                                          |
| 16                                | <b>End While</b>                                            |
| 17                                | <b>End</b>                                                  |

圖 15 MACS 路線建構之偽程式碼

#### 4.5.5 路線整合與評估

本節針對前述演算法所產生之配送路線進行整合與可行性驗證及效能評估。首先將主線與支援線之配送結果進行整合，形成完整之最終配送路線，再透過 Google Routes API 取得各條路線於實際道路網路環境下之行駛距離與預估行駛時間，以反映真實交通條件。將整合後之配送結果轉換為多項量化指標進行系統性評估，包括所需車輛數量、總行駛距離、平均車輛裝載率以及時間窗滿足比例等，以衡量整體配送效率與路線合理性。最後彙整最終配送路線資料及其對應之評估指標，作為後續不同規劃策略之比較分析，以及實際配送系統應用之參考依據。

## 4.6 實驗結果比較

在經過本研究所設計之自動化路線編排流程後，本節將評估該流程在物流公司提供之資料集上的路線編排效果，分析不同超參數設定對路線規劃結果之影響，並進一步檢視整體路線編排績效指標，以確認結果是否符合預期，從而驗證其在實務配送環境中的可行性與穩定性。此外，為更全面評估所提方法之效能，亦將進一步與基準方法進行比較分析，以檢視本研究方法在整體路線品質等面向之優勢。

### 4.6.1 符號定義

*Ins.*: 物流公司提供之資料實例編號

*NV*: 車輛總數量

*TD*: 總行駛距離

*U*: 路線平均裝載率

*O*: 準點率

### 4.6.2 實驗結果比較

本研究採用物流公司之實際人工編排路線資料作為比較基準，以每日配送需求為實驗單位，共 26 組實例。系統進行路線編排後，針對車輛數量、總行駛距離、路線平均裝載率、準點率及路線規劃所需時間進行量化分析，以評估所提方法之整體效能。

本實驗旨在驗證本研究所提方法於配送情境之成效，相關實驗數據彙整如表 6、表 7、表 8 所示。其中表 6 呈現各實例於物流公司人工編排以及基準方法之車輛數量、總行駛距離、平均裝載率及準點率等指標比較結果，表 7 進一步統計各評估指標之平均改善幅度，用以分析整體效益之變化趨勢。表 8 則比較物流公司人工編排與本研究提出方法以及基準方法在整體路線編排所需時間之差異，以評估系統在規劃效率方面之提升。

表 6 所提出方法與物流公司人工編排比較結果

| <i>Ins.</i> | Proposed Method |           |              |              | 物流公司人工編排路線 |           |              |              |
|-------------|-----------------|-----------|--------------|--------------|------------|-----------|--------------|--------------|
|             | <i>NV</i>       | <i>TD</i> | <i>U</i> (%) | <i>O</i> (%) | <i>NV</i>  | <i>TD</i> | <i>U</i> (%) | <i>O</i> (%) |
| D20221203   | 93.4            | 4,508.47  | 92.3         | 99.87        | 102        | 4,623.34  | 84.85        | 94.7         |
| D20221205   | 83.4            | 3,951.71  | 89.99        | 98.62        | 90         | 4,069.95  | 83.62        | 94.04        |
| D20221207   | 85.7            | 4,093.01  | 90.65        | 99.56        | 89         | 4,134.37  | 86.95        | 90.38        |
| D20221208   | 82              | 3,798.70  | 79.72        | 98.61        | 83         | 3,826.14  | 78.76        | 89.89        |
| D20221209   | 84.2            | 3,952.41  | 88.34        | 99.69        | 89         | 4,042.17  | 83.4         | 89.46        |
| D20221210   | 85.2            | 4,100.60  | 93           | 99.79        | 94         | 4,264.06  | 85.26        | 91.47        |
| D20221212   | 83.4            | 3,986.38  | 91.43        | 99.94        | 91         | 4,169.74  | 85.74        | 92.93        |
| D20221213   | 99.7            | 4,719.31  | 93.13        | 99.76        | 108        | 4,835.70  | 85.15        | 96.26        |
| D20221214   | 82.5            | 3,906.81  | 86.86        | 99.29        | 85         | 3,915.92  | 84.22        | 92.4         |
| D20221215   | 81              | 3,796.71  | 79.66        | 98.06        | 81         | 3,807.76  | 79.6         | 90.04        |
| D20221216   | 81              | 3,802.63  | 82.13        | 97.49        | 83         | 3,859.83  | 80.19        | 87.36        |
| D20221217   | 81              | 3,791.13  | 85.66        | 98.67        | 83         | 3,855.24  | 83.59        | 90.18        |
| D20221219   | 81              | 3,805.58  | 84.93        | 97.34        | 82         | 3,812.78  | 83.93        | 88           |
| D20221220   | 92.7            | 4,416.36  | 93.95        | 99.67        | 100        | 4,542.96  | 87.09        | 93.78        |
| D20221221   | 85.5            | 4,099.36  | 90.82        | 98.71        | 90         | 4,172.23  | 86.29        | 91.5         |
| D20221222   | 81.7            | 3,837.39  | 86.8         | 98.61        | 84         | 3,882.31  | 84.31        | 88.86        |
| D20221223   | 121.5           | 5,591.63  | 92.66        | 99.77        | 129        | 5,567.91  | 87.21        | 95.2         |
| D20221227   | 123             | 5,662.11  | 92.53        | 99.78        | 134        | 5,726.80  | 85.12        | 95.75        |
| D20221230   | 101.2           | 4,760.85  | 92.53        | 99.83        | 105        | 4,701.75  | 89.56        | 94.95        |
| D20221231   | 103.5           | 4,907.46  | 92.27        | 99.95        | 111        | 4,946.66  | 86.76        | 94.13        |
| D20230102   | 103.7           | 4,973.15  | 92.51        | 99.74        | 110        | 4,883.74  | 87.05        | 96.52        |
| D20230107   | 98.7            | 4,716.04  | 93.13        | 99.83        | 120        | 5,373.11  | 76.9         | 91.75        |

|           |       |          |       |       |     |          |       |       |
|-----------|-------|----------|-------|-------|-----|----------|-------|-------|
| D20230113 | 83.2  | 3,919.75 | 88.24 | 98.44 | 91  | 4,168    | 80.88 | 91.96 |
| D20260102 | 99.5  | 4,606.42 | 91.9  | 98.17 | 136 | 5,553.82 | 67.91 | 94.15 |
| D20260106 | 128.7 | 6,011.78 | 92.3  | 99.74 | 159 | 6,891.41 | 74.62 | 93.11 |
| D20260109 | 88.8  | 3,996.55 | 83.74 | 95.82 | 94  | 4,191.59 | 79.06 | 94.27 |

實驗結果顯示，本研究方法能有效提升配送資源利用效率並降低配送成本。相比於物流公司人工編排結果，在車輛數方面，26 個實例中有 25 個實例獲得改善，平均減少 7% 車輛需求，最高可降低 26.75%，顯示本研究方法能有效減少所需配送車輛數量。在總配送距離方面，大部分實例皆呈現下降趨勢，平均降低 3.07%，最高可降低 17.09%，表示本研究方法在減少車輛使用的同時，仍能維持較佳之路線規劃效率。此外，所有實例平均裝載率皆有所提升，平均增加 6.7%，最高提升 35.92%。準點率方面，皆獲得改善平均提高 7.16%，最高提升 10.96%。在編排時間方面，人工編排大約需 1 小時至 3 小時，而本研究方法大約需 2 分鐘至 30 分鐘，可將原本需數小時的配送規劃縮短至數分鐘，大幅降低規劃時間成本並提升作業效率。綜合上述結果可知，本研究所提出之配送規劃方法能有效降低整體物流成本，並提升整體配送系統之運作效率與服務品質。

表 7 所提出方法平均改善率

| 指標 | 平均改善率  |
|----|--------|
| NV | -7.00% |
| TD | -3.07% |
| U  | +6.70% |
| O  | +7.16% |

表 8 系統與物流公司人工編排平均編排時間

| 指標   | Proposed Method | 人工編排    |
|------|-----------------|---------|
| 編排時間 | 2-30 mins       | 1-3 hrs |

從圖 16、圖 17 可知，各實例因應每日訂單需求的波動，其需處理的店鋪數量存在明顯差異，多數介於 20 至 100 家之間。由兩圖結果可知，編排時間與店鋪數量呈現明顯正相關。大部分實例的編排時間可控制於 5 至 15 分鐘內。然而，當需處理的店鋪數量增加時，進行路徑編排與空間搜尋時所需的運算成本便會成比例增加。由於需抽取之店鋪數量與原始路線的超載程度具有高度相關，因此對應資料的路線平均超載率將間接影響本研究方法的運算效率。

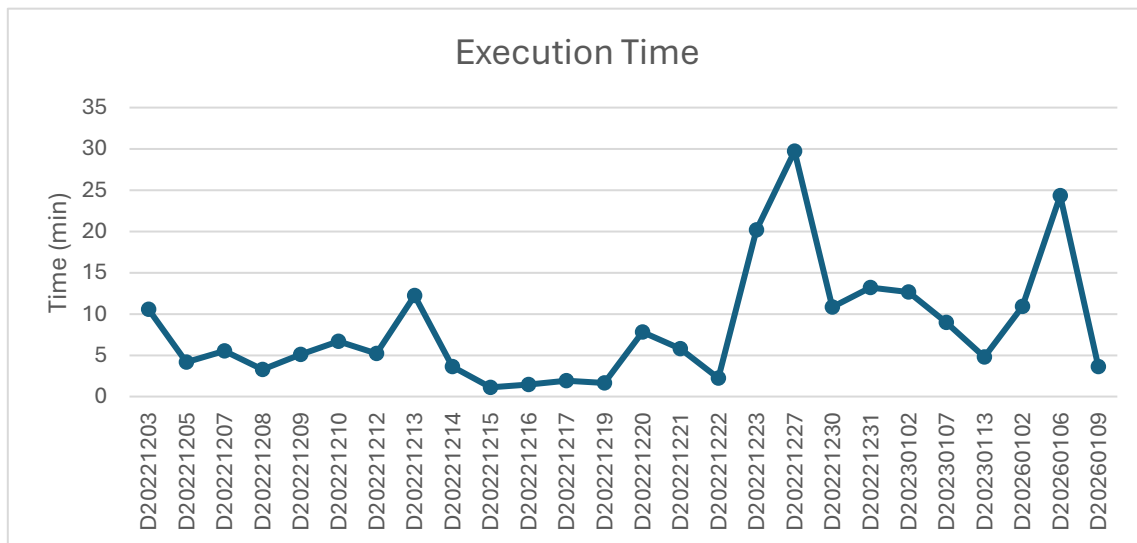


圖 16 所提出方法各實例之編排時間



圖 17 所提出方法各實例於店鋪抽取階段之抽取店鋪數量



## 4.6.3 實例研究與分析

### 4.6.3.1 路線分析

為進一步探討改善效果較為顯著之原因，本研究選取 D20260102 進行實例分析，該實例於車輛需求方面降低 26.75%，顯示其原始配送規劃仍具備較大優化空間，且路線結構差異較為明顯，具代表性與分析價值。因此，以下將先說明該實例之整體配送結構變化，並進一步以具代表性之 2M 路線為例，探討節點重組與路線整併對配送效率之影響。

由表 9 可知，系統規劃支援線車輛數由 50 輛降低至 10 輛，顯示由支援線執行之配送任務已轉移至主線車輛承擔，並有效降低額外支援車輛之需求。主線店鋪數量由 651 間增加至 724 間，而支援線店鋪數量則由 118 間下降至 45 間，顯示，在部分路線負載超過上限之情況下，系統透過跨路線背載機制，將更多配送需求被重新分配至主線路線執行，使配送需求得以重新分配並維持整體可行性。此外，在車輛利用率方面，主線裝載率由 0.84 提升至 0.94，支援線平均裝載率由 0.40 提升至 0.93，顯示支援車輛已被有效整併並提升其載運效能。

表 9 D20260102 整體配送結果比較表

| 指標       | 人工編排     | Proposed Method |
|----------|----------|-----------------|
| 主線數量     | 86       | 88              |
| 支援線數量    | 50       | 10              |
| 主線店鋪數量   | 651      | 724             |
| 支援線店鋪數量  | 118      | 45              |
| 主線總行駛距離  | 3,833.61 | 4159.19         |
| 支援線總行駛距離 | 1,720.18 | 416.19          |
| 主線裝載率    | 0.84     | 0.94            |
| 支援線裝載率   | 0.40     | 0.93            |

為說明路線背載對配送效率之影響，本研究選取 D20260102 的 2M 路線作為代表性實例進行分析。由表 10 可知，人工編排之 2M 路線總材積量為 3.31，裝載率為 0.46，總行駛距離為 47.35 公里。相較之下，系統規劃後之 2M 路線總材積量提升至 6.98，裝載率提升至 0.97，總行駛距離為 50.44 公里。此結果反映該等店鋪原本位於其他路線中，因原路線負載已超出容量限制，透過跨路線背載機制，將部分配送需求分配至 2M 路線承擔，使其提升主線運輸效率。

表 10 D20260102 路線 2M 摘要比較表

| 方法              | 總材積量 | 裝載率  | 總行駛距離 |
|-----------------|------|------|-------|
| 人工編排            | 3.31 | 0.46 | 47.35 |
| Proposed Method | 6.98 | 0.97 | 50.44 |

由表 11 可知，人工編排之 2M 路線共服務 5 間店鋪，分別為機場捷運店、高鐵二店、台鐵一店、北車捷運店及松江南京店，其總材積量為 3.31。路線總裝載率僅為 0.46，顯示該路線仍存在明顯之載運容量未充分利用情形。

表 11 D20260102 手動編排之路線 2M 店鋪資訊表

| 店鋪編號 | 店鋪名稱  | 材積量  |
|------|-------|------|
| 2M01 | 機場捷運店 | 0.7  |
| 2M02 | 高鐵二店  | 0.66 |
| 2M03 | 台鐵一店  | 0.81 |
| 2M04 | 北車捷運店 | 0.69 |
| 2M05 | 松江南京店 | 0.45 |
| 總材積量 | 3.31  |      |

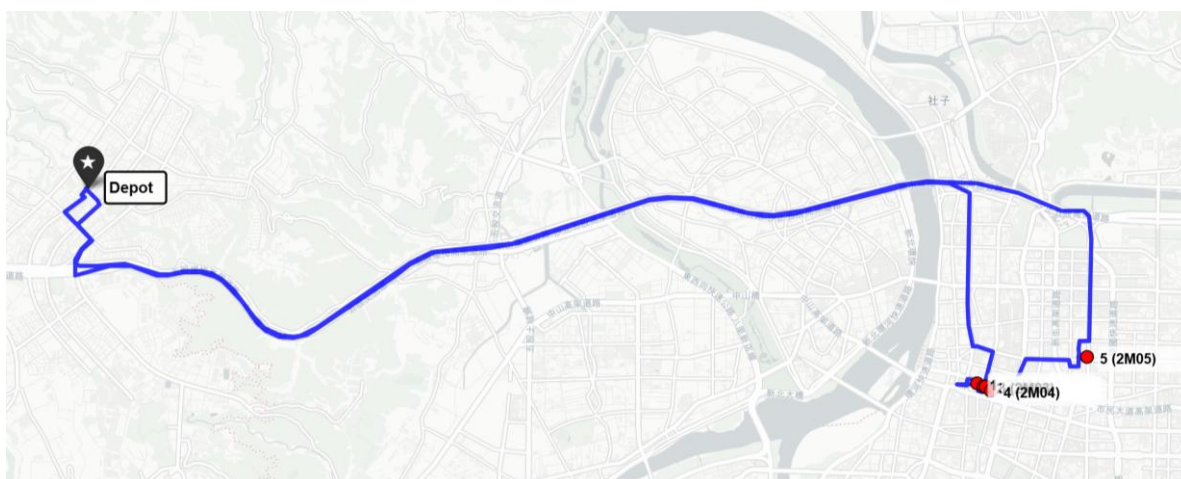


圖 18 D20260102 手動編排之路線 2M 結果

相較之下，由表 12 可知，系統規劃後於原 2M 路線中新增五股維興店、台鐵西店、台鐵北店及錦江店等 4 間店鋪，使服務店鋪數提升至 9 間，總材積量增加至 6.98，裝載率亦提升至 0.97。此結果顯示，透過跨路線背載機制重新分配至 2M 路線，使原有路線之剩餘容量得以有效利用，並提升整體配送負載平衡性。

表 12 D20260102 系統編排之路線 2M 店鋪資訊表

| 店鋪編號 | 店鋪名稱  | 材積量  |
|------|-------|------|
| 4J02 | 五股維興店 | 0.62 |
| 2M01 | 機場捷運店 | 0.7  |
| 3I05 | 台鐵西店  | 0.73 |
| 2M02 | 高鐵二店  | 0.66 |
| 2M03 | 台鐵一店  | 0.81 |
| 3I06 | 台鐵北店  | 1.5  |
| 2M04 | 北車捷運店 | 0.69 |
| 2M05 | 松江南京店 | 0.45 |
| 4A05 | 錦江店   | 0.82 |
| 總材積量 | 6.98  |      |

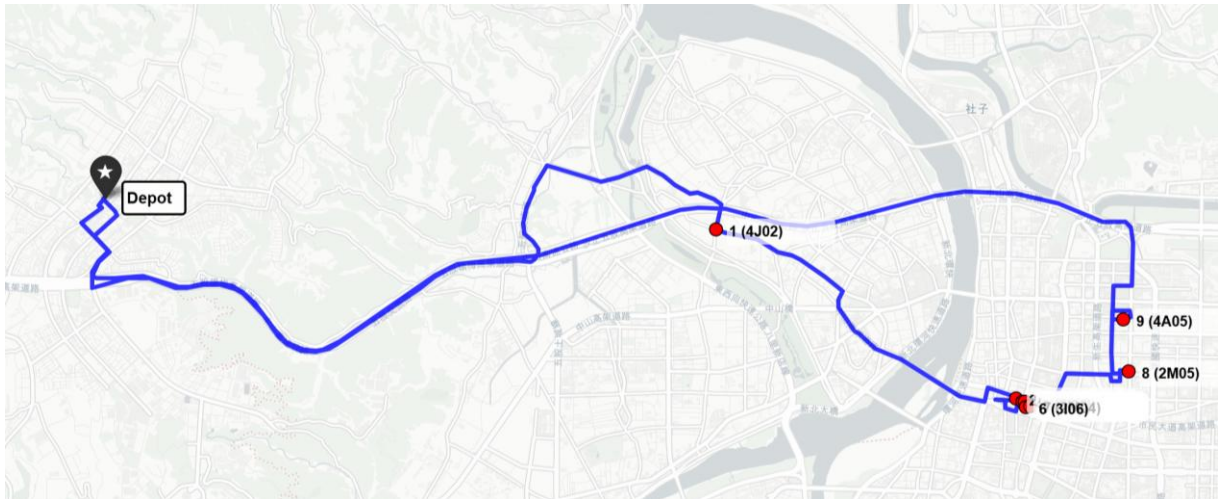


圖 19 D20260102 系統編排之路線 2M 結果

#### 4.6.3.2 收斂分析

本節分別針對 CCGA、HACO 與 MACS 於搜尋過程中的最佳解收斂曲線進行分析，以探討各階段的收斂行為與穩定性。圖 20 顯示 CCGA 在演化初期即可快速降低適應度值，能在較少的世代數內找到品質較佳的解。然而隨著世代增加，收斂速度逐漸趨緩，並於後期進入穩定狀態，顯示已收斂至局部或近似全域最佳解。

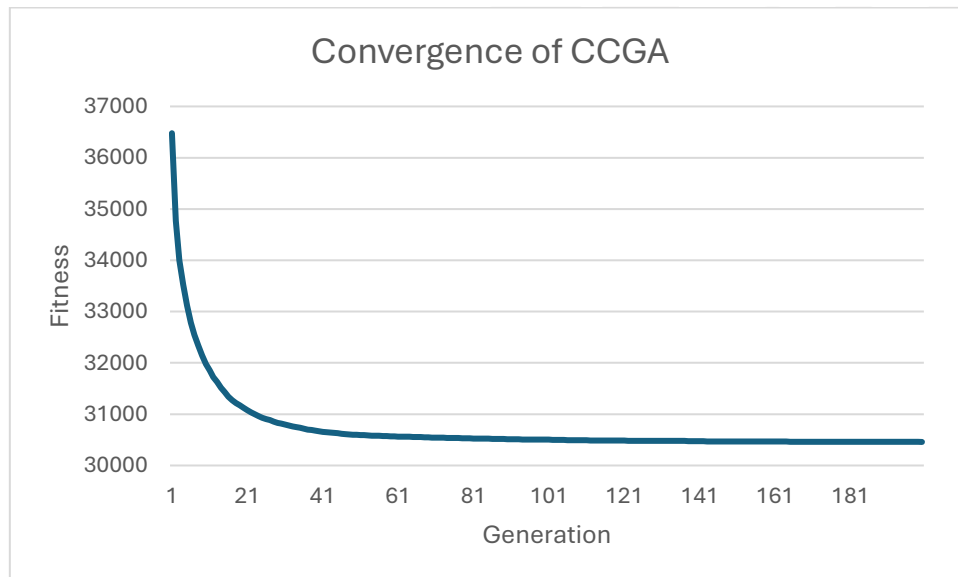


圖 20 CCGA 之適應度收斂曲線圖

圖 21 顯示店鋪分配階段 HACO 之收斂曲線，與其他階段方法相比，其收斂曲線並非呈現平滑下降，而是呈現階梯式改善。此現象主要源自於 ACO 在路徑構建過程中的隨機性與各類限制條件的影響，並受到費洛蒙更新與局部搜尋所造成之結果。

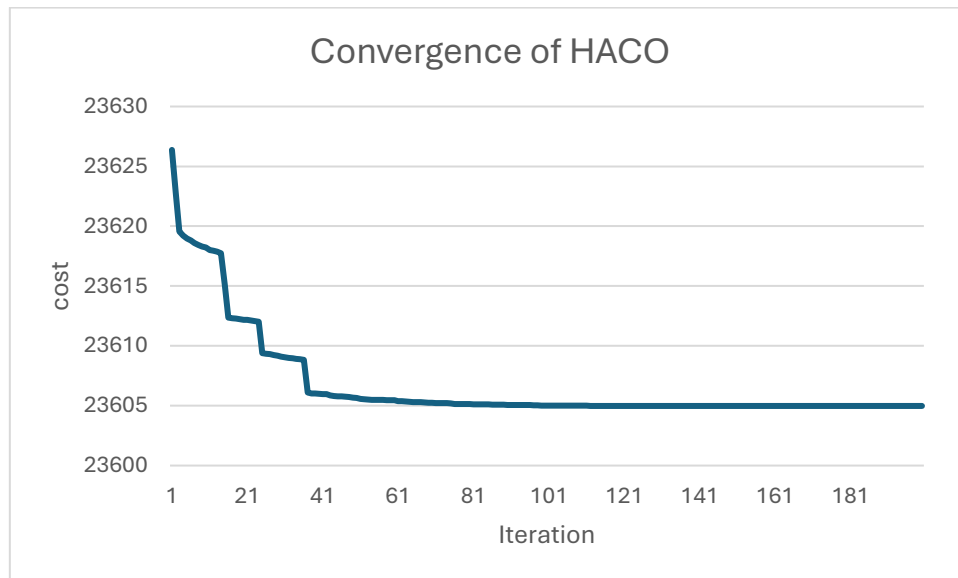


圖 21 HACO 之成本收斂曲線圖

圖 22 顯示 MACS 之收斂曲線，其在初期即快速下降，並於後續迭代中持續進行小幅改善，顯示其在整體搜尋過程中具備穩定持續優化的能力。

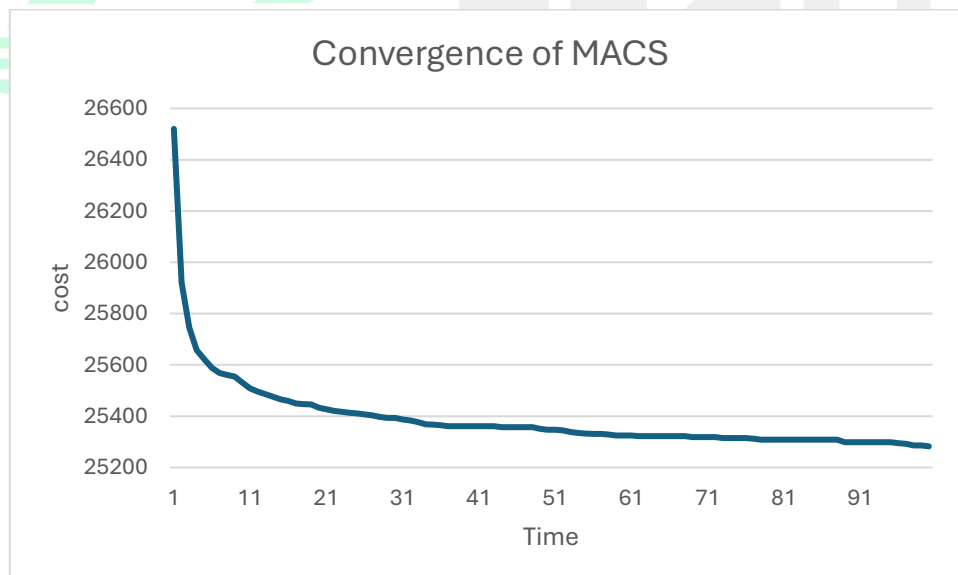


圖 22 MACS 之成本收斂曲線圖

#### 4.6.4 消融實驗與比較分析

本研究針對多階段配送規劃架構中各機制進行消融實驗，評估方法於配送結果上的實際貢獻。本研究共設計三種消融實驗，包括 Random Extraction、HACO without VND 及 MACS without VND，並於相同測試實例與參數設定下進行比較，以確保實驗之公平性。此外，另以 Single-Stage GA 作為單階段最佳化基準方法，比較多階段規劃架構與單階段方法之結果差異。透過上述比較，可進一步驗證各模組及多階段規劃架構對配送結果之貢獻。

表 13 實驗比較方法說明

| 方法                | 說明                          |
|-------------------|-----------------------------|
| Single-Stage GA   | 採用 HGA-SIH[13]單階段作為基準方法比較   |
| Random Extraction | 以隨機抽取方式取代 CCGA 候選店鋪選取機制     |
| HACO Without VND  | 主線規劃階段 HACO 不結合 VND 局部搜尋策略  |
| MACS Without VND  | 支援線規劃階段 MACS 不結合 VND 局部搜尋策略 |
| Proposed Method   | 完整多階段配送路線規劃框架               |

從圖 23、圖 24、圖 25、圖 26 結果可知，本研究之多階段最佳化方法相較 Single-Stage GA 於大多數測試案例中皆能在各項指標取得較佳之結果，顯示透過將問題分解為多個子階段，並於各階段採用適當之最佳化方法，相較於單一階段直接求解，能更有效降低配送成本並提升整體規劃品質。當比較完整方法與三種消融方法時，由於各方法皆建立於相同的多階段規劃架構之上，其整體趨勢相當接近，僅依據圖形結果較難客觀判斷各模組對整體配送績效的實際貢獻。因此，為進一步驗證完整方法相較於各消融方法之改善效果是否具有統計顯著性，本研究進一步針對各項指標進行統計檢定分析，藉以確認各模組對整體配送結果之影響程度，並評估多階段規劃架構及各最佳化方法於實務配送問題中的實際貢獻。



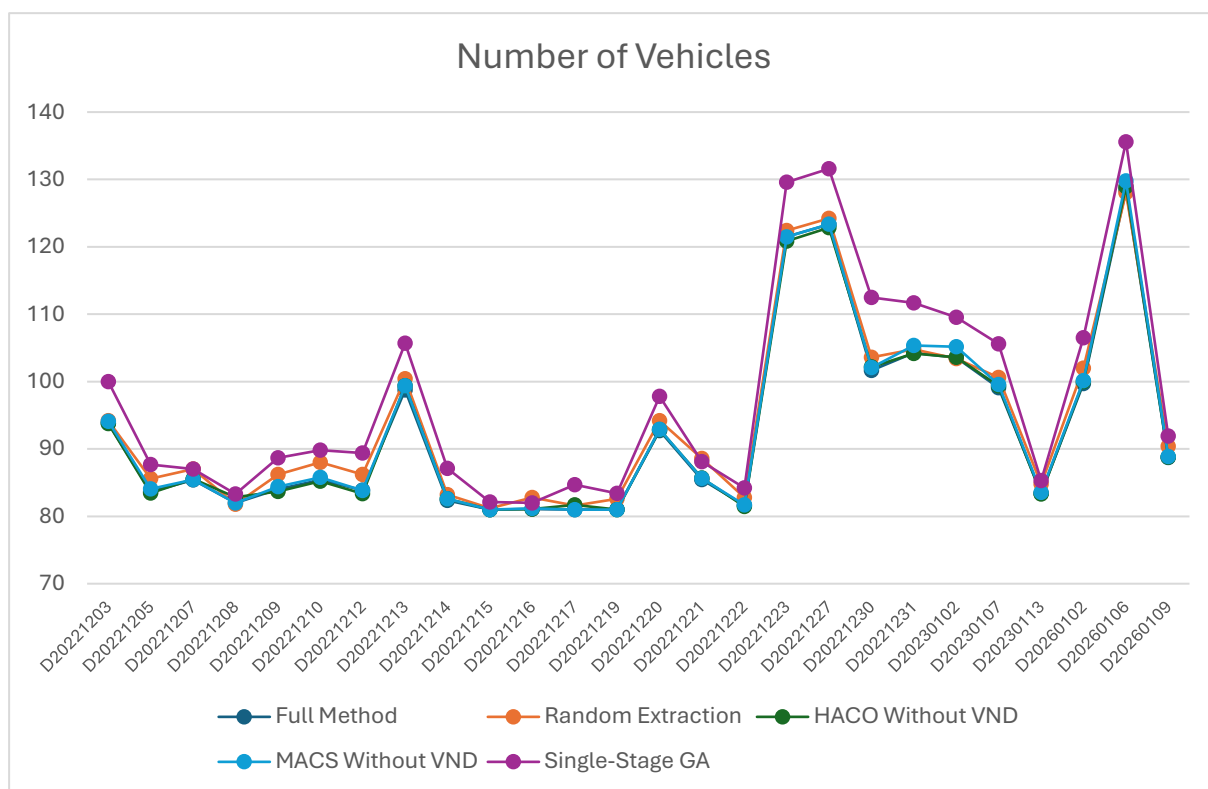


圖 23 消融實驗於各實例之總車輛數量比較結果

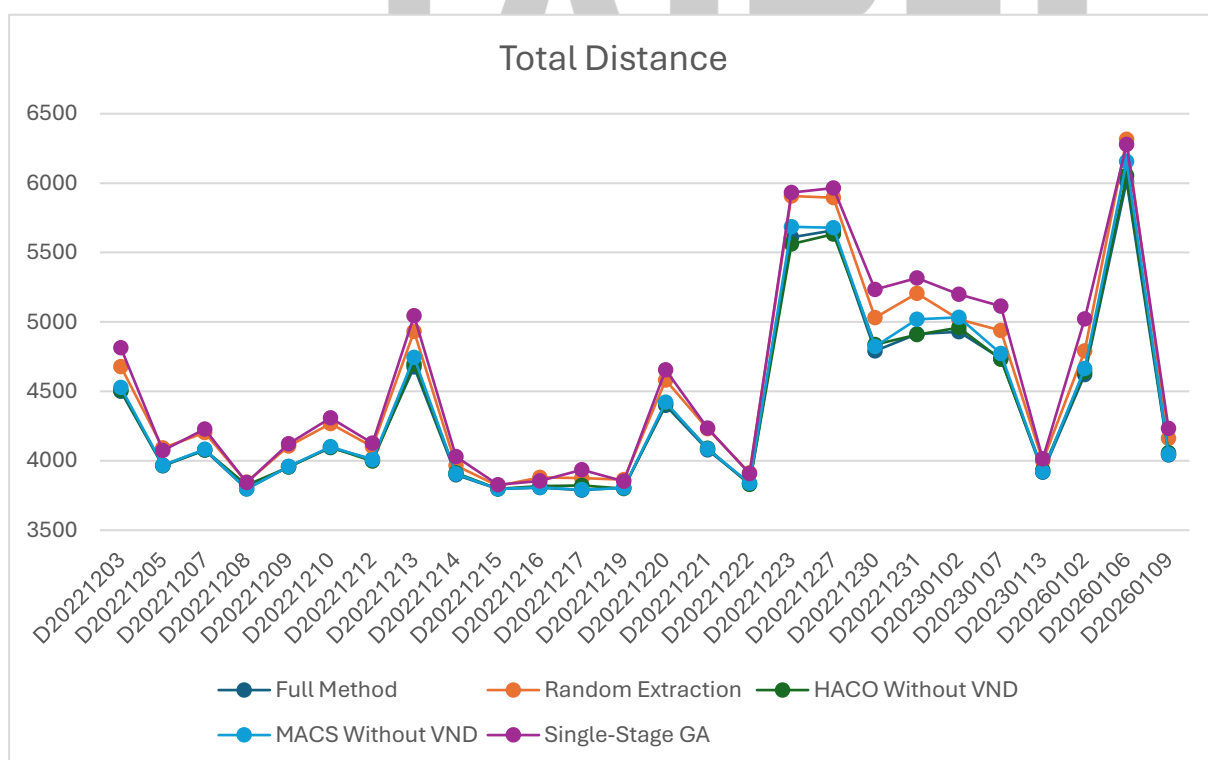


圖 24 消融實驗於各實例之總行駛距離比較結果



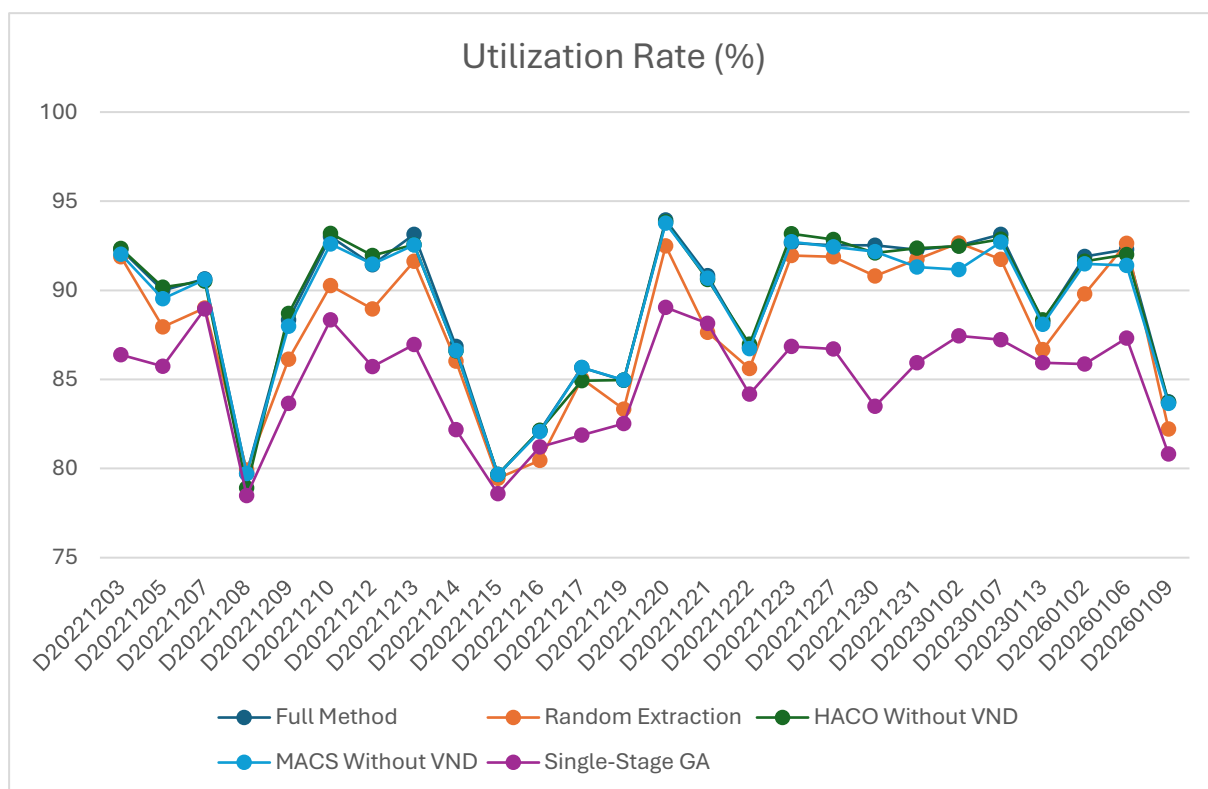


圖 25 消融實驗於各實例之車輛裝載率比較結果

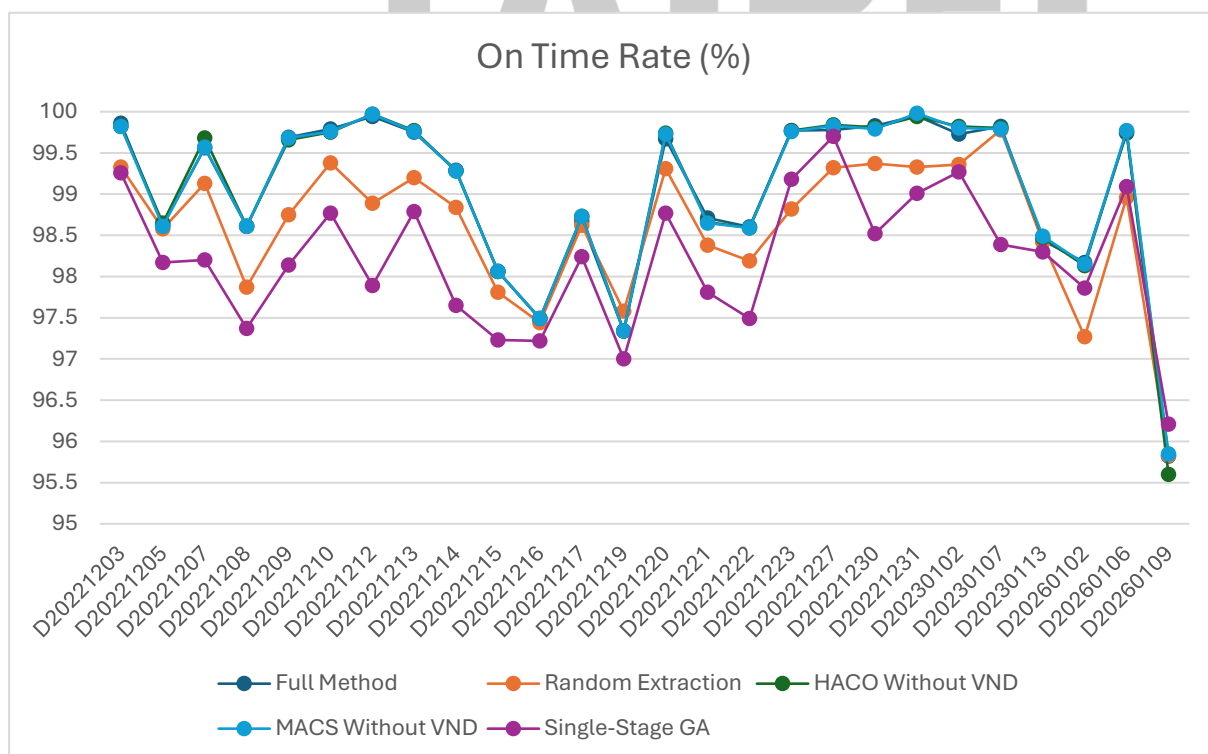


圖 26 消融實驗於各實例之店鋪準點率比較結果

本研究首先採用 Friedman test 進行比較分析，透過比較各方法於不同實例中的平均排名，檢驗各方法之整體表現是否存在顯著差異。顯著水準設定為 0.05。由表 14 可知，各方法於五項指標之平均排名存在明顯差異。同時，本研究提出之多階段配送規劃架構於總車輛數量、總行駛距離、車輛平均裝載率皆獲得最佳平均排名，顯示其在配送效率及資源利用方面具有最佳整體表現，在準點率方面則略差於 MACS Without VND 之結果，僅差 0.20，顯示加入 VND 對準點率影響有限，但能有效提升其他配送績效指標。

表 14 Friedman 檢定之方法平均排名結果

| 方法                | NV   | TD   | U    | O    |
|-------------------|------|------|------|------|
| Single-Stage GA   | 4.9  | 4.77 | 4.88 | 4.38 |
| Random Extraction | 3.62 | 4.19 | 3.62 | 3.81 |
| HACO Without VND  | 2.1  | 2.12 | 2.15 | 2.27 |
| MACS Without VND  | 2.52 | 2.38 | 2.42 | 2.17 |
| Proposed Method   | 1.87 | 1.54 | 1.92 | 2.37 |

由表 15 之統計檢定結果可知，各指標 p-value 值均小於 0.05，表示各比較方法間存在統計顯著差異，顯示不同方法確實會對配送結果產生影響。

表 15 Friedman 統計檢定結果

| 方法                 | NV       | TD       | U        | O        |
|--------------------|----------|----------|----------|----------|
| Friedman Statistic | 68.15    | 81.63    | 76.5     | 67.64    |
| p-value            | 5.57E-14 | 7.86E-17 | 9.61E-16 | 7.14E-14 |

為進一步驗證所提出方法相較於比較方法之改善是否具有統計顯著性，本研究針對各測試實例進行成對比較。由於各配送實例在不同方法下具有相同需求條件，且最佳化結果不一定符合常態分布，因此採用 Wilcoxon Signed-Rank Test 對所提出方法與各比較進行統計檢定。顯著水準同樣設定為 0.05。

由表 16 可知，本研究方法相比於單階段方法於各指標均取得較佳結果，且均達到

顯著差異，其原因為本研究採用多階段最佳化架構，可有效限縮解空間，提升搜尋效率與解的品質。另一方面，相較於隨機抽取方法，本研究亦於各項評估指標取得較佳結果且達到顯著差異。此外，在 HACO Without VND 比較中，顯著差異主要出現在總行駛距離指標上，此結果顯示 VND 作為局部搜尋機制有助於改善路線內部之行駛效率。而在 MACS Without VND，結果顯示，VND 能有效降低車輛數量及總行駛距離，並且進一步提高平均裝載率，顯示其於路線方面具進一步之改善效果。

表 16 Wilcoxon Signed-Rank Test 成對比較結果表

| 方法                            | 指標 | W     | p-value  | Significance |
|-------------------------------|----|-------|----------|--------------|
| Proposed vs Single-Stage GA   | NV | 0     | 2.98E-08 | Yes          |
|                               | TD | 0     | 2.98E-08 | Yes          |
|                               | U  | 0     | 2.98E-08 | Yes          |
|                               | O  | 0     | 4.15E-05 | Yes          |
| Proposed vs Random Extraction | NV | 13.5  | 2.62E-06 | Yes          |
|                               | TD | 0     | 2.98E-08 | Yes          |
|                               | U  | 15    | 7.64E-05 | Yes          |
|                               | O  | 9.5   | 1.62E-04 | Yes          |
| Proposed vs HACO Without VND  | NV | 124.5 | 4.66E-01 | No           |
|                               | TD | 81    | 1.51E-02 | Yes          |
|                               | U  | 8     | 2.56E-01 | No           |
|                               | O  | 0     | 3.17E-01 | No           |
| Proposed vs MACS Without VND  | NV | 36    | 5.71E-03 | Yes          |
|                               | TD | 54    | 1.29E-03 | Yes          |
|                               | U  | 0     | 1.40E-02 | Yes          |
|                               | O  | 2     | 1.57E-01 | No           |

#### 4.6.5 與基準方法之比較

為驗證本研究所提出方法之有效性，本文選擇相關文獻中具代表性之方法作為基準進行比較，並於 Solomon VRPTW Benchmark 測試資料上進行實驗，並於相同評估指標下分析各方法之表現差異。所採用之測試資料之節點數量皆為 100 個，並依據客戶分布特性可分為 C、R 與 RC 三類，其中 C 類為群聚型(Clustered)，客戶節點呈現明顯群聚分布，R 類為隨機型(Random)，客戶位置均勻隨機分布，RC 類則為混合型(Random-Clustered)，同時包含隨機與群聚特性，以評估不同空間分布條件下之方法表現。評估指標主要包含車輛數與總行駛距離，以同時衡量解的品質與實務應用中的運輸成本效益。在實驗設計上，本研究依據 baseline 論文設定，對每一測試實例均重複執行 30 次，並以其比較準則選取代表解進行比較。首先以最小化車輛數量為主要目標，再於相同車輛數條件下比較總行駛距離，並以此作為最終結果進行比較。

表 17 本研究與基準方法於 C 類之比較

| Ins. | Baseline |        | Best Known |        | Proposed Method |        |                      |                      |
|------|----------|--------|------------|--------|-----------------|--------|----------------------|----------------------|
|      | NV       | TD     | NV         | TD     | NV              | TD     | Gap <sup>V</sup> (%) | Gap <sup>D</sup> (%) |
| C101 | 10       | 828.94 | 10         | 828.94 | 10              | 828.94 | 0                    | 0                    |
| C102 | 10       | 828.94 | 10         | 828.94 | 10              | 828.94 | 0                    | 0                    |
| C103 | 10       | 830.77 | 10         | 828.06 | 10              | 828.06 | 0                    | -0.33                |
| C104 | 10       | 864.22 | 10         | 824.78 | 10              | 826.37 | 0                    | -4.38                |
| C105 | 10       | 828.94 | 10         | 828.94 | 10              | 828.94 | 0                    | 0                    |
| C106 | 10       | 828.94 | 10         | 828.94 | 10              | 828.94 | 0                    | 0                    |
| C107 | 10       | 828.94 | 10         | 828.94 | 10              | 828.94 | 0                    | 0                    |
| C108 | 10       | 854.31 | 10         | 828.94 | 10              | 828.94 | 0                    | -2.97                |
| C109 | 10       | 854.78 | 10         | 828.94 | 10              | 828.94 | 0                    | -3.02                |
| C201 | 3        | 591.56 | 3          | 591.56 | 3               | 591.56 | 0                    | 0                    |
| C202 | 3        | 591.56 | 3          | 591.56 | 3               | 591.56 | 0                    | 0                    |
| C203 | 3        | 591.17 | 3          | 591.17 | 3               | 591.17 | 0                    | 0                    |
| C204 | 3        | 594.51 | 3          | 590.60 | 3               | 612.39 | 0                    | 3.01                 |
| C205 | 3        | 588.88 | 3          | 588.85 | 3               | 588.88 | 0                    | 0                    |

|      |   |        |   |        |   |        |   |   |
|------|---|--------|---|--------|---|--------|---|---|
| C206 | 3 | 588.49 | 3 | 588.49 | 3 | 588.49 | 0 | 0 |
| C207 | 3 | 588.29 | 3 | 588.29 | 3 | 588.29 | 0 | 0 |
| C208 | 3 | 588.32 | 3 | 588.32 | 3 | 588.32 | 0 | 0 |

在 C 類實例結果中，本研究所提出之方法於所有實例皆可取得與最佳已知解相同之車輛數量。此外，在多數實例中，本方法之總行駛距離亦可達到最佳已知解，部分實例中優於基準方法。例如於 C103 實例中，本方法成功取得與最佳已知解相同之結果，相較基準方法降低 0.33% 之行駛距離，於 C104、C108 與 C109 實例中，本方法亦分別較基準方法降低 4.38%、2.97% 與 3.02% 之總行駛距離。然而，在部分實例中，本方法之總行駛距離仍與最佳已知解存在差距，例如 C104 與 C204 實例，其距離分別高於最佳已知解約 0.19% 與 3.69%。

表 18 本研究與基準方法於 R 類之比較

| <i>Ins.</i> | Baseline  |           | Best Known |           | Proposed Method |           |                           |                           |
|-------------|-----------|-----------|------------|-----------|-----------------|-----------|---------------------------|---------------------------|
|             | <i>NV</i> | <i>TD</i> | <i>NV</i>  | <i>TD</i> | <i>NV</i>       | <i>TD</i> | <i>Gap<sup>V</sup>(%)</i> | <i>Gap<sup>D</sup>(%)</i> |
| R101        | 19        | 1656.55   | 19         | 1650.8    | 19              | 1653.53   | 0                         | -0.18                     |
| R102        | 18        | 1476.85   | 17         | 1486.12   | 17              | 1497.23   | -5.56                     | 1.38                      |
| R103        | 14        | 1230.07   | 13         | 1292.68   | 13              | 1322.8    | -7.14                     | 7.54                      |
| R104        | 10        | 1010.55   | 9          | 1007.31   | 10              | 1015.76   | 0                         | 0.52                      |
| R105        | 14        | 1395.68   | 14         | 1377.11   | 14              | 1436.85   | 0                         | 2.95                      |
| R106        | 13        | 1261.61   | 12         | 1252.03   | 12              | 1324.48   | -7.69                     | 4.98                      |
| R107        | 11        | 1103.77   | 10         | 1104.66   | 11              | 1106.51   | 0                         | 0.25                      |
| R108        | 10        | 966.99    | 9          | 960.88    | 10              | 983.23    | 0                         | 1.68                      |
| R109        | 12        | 1200.20   | 11         | 1194.73   | 12              | 1214.84   | 0                         | 1.22                      |
| R110        | 11        | 1127.28   | 10         | 1118.84   | 11              | 1119.33   | 0                         | -0.71                     |
| R111        | 11        | 1096.38   | 10         | 1096.72   | 11              | 1138.82   | 0                         | 3.87                      |
| R112        | 9         | 982.14    | 9          | 982.14    | 10              | 1005.83   | 11.11                     | 2.41                      |
| R201        | 4         | 1442.28   | 4          | 1252.37   | 4               | 1302.83   | 0                         | -9.67                     |
| R202        | 4         | 1212.49   | 3          | 1191.7    | 3               | 1406.77   | -25                       | 16.02                     |
| R203        | 3         | 1197.67   | 3          | 939.5     | 3               | 984.76    | 0                         | -17.78                    |
| R204        | 10        | 854.31    | 2          | 825.52    | 3               | 779.27    | -70                       | -8.78                     |
| R205        | 4         | 1337.65   | 3          | 994.43    | 3               | 1071.58   | -25                       | -19.89                    |

|      |   |         |   |        |   |        |        |        |
|------|---|---------|---|--------|---|--------|--------|--------|
| R206 | 4 | 1114.76 | 3 | 906.14 | 3 | 972.48 | -25    | -12.76 |
| R207 | 4 | 1055.71 | 2 | 890.61 | 3 | 848.77 | -25    | -19.6  |
| R208 | 3 | 860.53  | 2 | 726.82 | 2 | 814.22 | -33.33 | -5.38  |
| R209 | 2 | 895.47  | 3 | 909.16 | 3 | 965.75 | 50     | 7.85   |
| R210 | 2 | 942.95  | 3 | 939.37 | 3 | 994.83 | 50     | 5.5    |
| R211 | 3 | 1049.62 | 2 | 885.71 | 3 | 808.38 | 0      | -22.98 |

在 R 類實例結果中，本研究方法相較於基準方法，在多數實例中能有效降低車輛數量。於 R102、R103 與 R106 實例中，本方法皆較基準方法減少 1 輛車；其中 R103 與 R106 分別降低 7.14%與 7.69%之車輛數。於 R2 類實例中，此改善效果更為明顯，例如 R204 實例中車輛數由 10 輛降低至 3 輛，R205、R206 與 R207 亦皆由 4 輛降低至 3 輛，顯示本方法能有效提升車輛利用率並減少支援車輛需求。除改善車輛數量外，部分實例之總行駛距離亦優於基準方法。例如 R101、R110 與 R201 實例中，本方法於維持相同車輛數下，總距離分別降低 0.18%、0.71%與 9.67%；於 R204、R205、R206、R207 與 R211 實例中，不僅降低車輛數，同時有效減少總行駛距離。

然而，在部分實例中，雖然成功降低車輛數，但總行駛距離略有增加，例如 R102、R103 與 R106 實例。此結果顯示本方法在優先減少車輛使用數之目標下，可能透過增加部分行駛距離以提升路線整合程度。整體而言，考量 VRPTW 以最小化車輛數為主要目標，本研究方法於 R 類實例中相較於基準方法仍展現較佳之整體表現。

表 19 本研究與基準方法於 RC 類之比較

| Ins.  | Baseline |         | Best Known |         | Proposed Method |         |                      |                      |
|-------|----------|---------|------------|---------|-----------------|---------|----------------------|----------------------|
|       | NV       | TD      | NV         | TD      | NV              | TD      | Gap <sup>V</sup> (%) | Gap <sup>D</sup> (%) |
| RC101 | 16       | 1656.59 | 14         | 1696.95 | 15              | 1649.47 | -6.25                | -0.43                |
| RC102 | 13       | 1532.25 | 12         | 1554.75 | 13              | 1497.51 | 0                    | -2.27                |
| RC103 | 12       | 1344.03 | 11         | 1261.67 | 11              | 1316.11 | -8.33                | -2.08                |
| RC104 | 11       | 1184.90 | 10         | 1135.48 | 10              | 1169.1  | -9.09                | -1.33                |
| RC105 | 14       | 1662.01 | 13         | 1629.44 | 14              | 1595.23 | 0                    | -4.02                |
| RC106 | 12       | 1344.03 | 11         | 1424.73 | 12              | 1424.21 | 0                    | 5.97                 |
| RC107 | 12       | 1263.07 | 11         | 1230.48 | 11              | 1262.21 | -8.33                | -0.07                |

|       |    |         |    |         |    |         |     |       |
|-------|----|---------|----|---------|----|---------|-----|-------|
| RC108 | 11 | 1144.94 | 10 | 1139.82 | 11 | 1140.16 | 0   | -0.42 |
| RC201 | 4  | 1442.28 | 4  | 1406.94 | 4  | 1495.19 | 0   | 3.67  |
| RC202 | 4  | 1212.49 | 3  | 1365.65 | 4  | 1209.52 | 0   | -0.24 |
| RC203 | 3  | 1197.67 | 3  | 1049.62 | 3  | 1136.33 | 0   | -5.12 |
| RC204 | 10 | 854.31  | 3  | 798.46  | 3  | 823.83  | -70 | -3.57 |
| RC205 | 4  | 1337.65 | 4  | 1297.65 | 4  | 1374.6  | 0   | 2.76  |
| RC206 | 4  | 1114.76 | 3  | 1146.32 | 3  | 1258.08 | -25 | 12.86 |
| RC207 | 4  | 1055.71 | 3  | 1061.14 | 3  | 1160.47 | -25 | 9.92  |
| RC208 | 3  | 860.53  | 3  | 828.14  | 3  | 883.61  | 0   | 2.68  |

在 RC 類實例中，本研究方法相較於基準方法，多數實例皆能降低或維持相同車輛數，其中 RC103、RC104 與 RC107 實例成功減少 1 輛車，RC204、RC206 與 RC207 實例則分別降低 70%、25%與 25%之車輛使用數。在總行駛距離方面，本研究方法於多數 RC1 實例中皆優於基準方法，例如 RC101、RC102、RC103、RC104 與 RC105 實例之總距離皆有所降低。然而，部分 RC2 實例雖成功降低車輛數，但總行駛距離略有增加，例如 RC206 與 RC207 實例。此結果顯示本方法在優先降低車輛數之目標下，可能透過增加部分行駛距離以提升整體配送整合效果。

本研究方法與 Baseline 方法於各類測試資料中的車輛數與總行駛距離表現比較結果顯示，在多數實例中皆能取得相近或更佳之結果，顯示其具有良好的路線規劃能力與搜尋效率。在車輛數方面，本研究於 R2 與 RC2 類型資料中成功降低車輛使用數，代表其在顧客分布較分散或混合型情境下，能有效提升車輛裝載率與路線整合能力。在總行駛距離方面，於 R2 類型資料中改善較為明顯。結果顯示，本研究方法在不同類型測試資料下皆具備穩定且具競爭力之最佳化能力，如圖 27 與圖 28 所示。



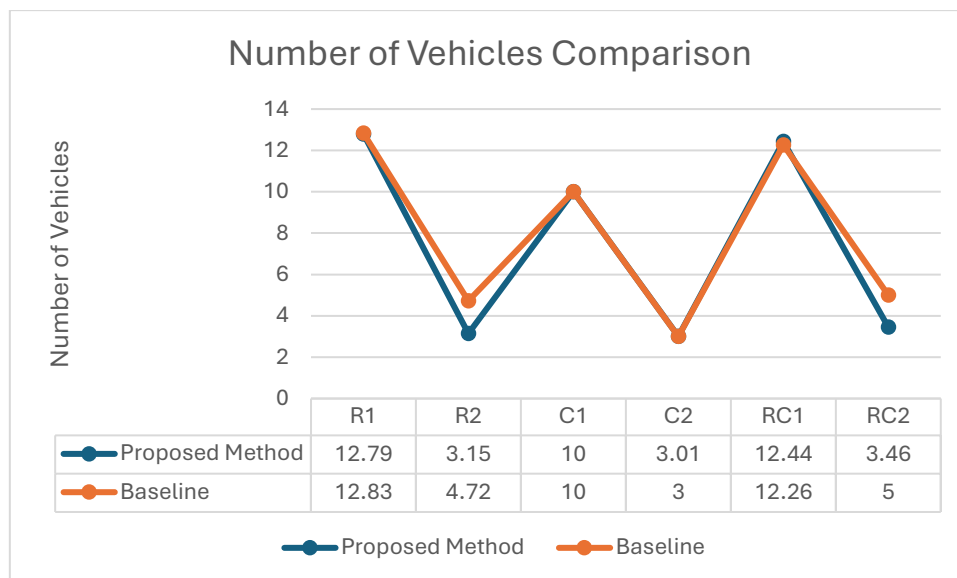


圖 27 本研究與基準方法於各類資料之車輛數量比較

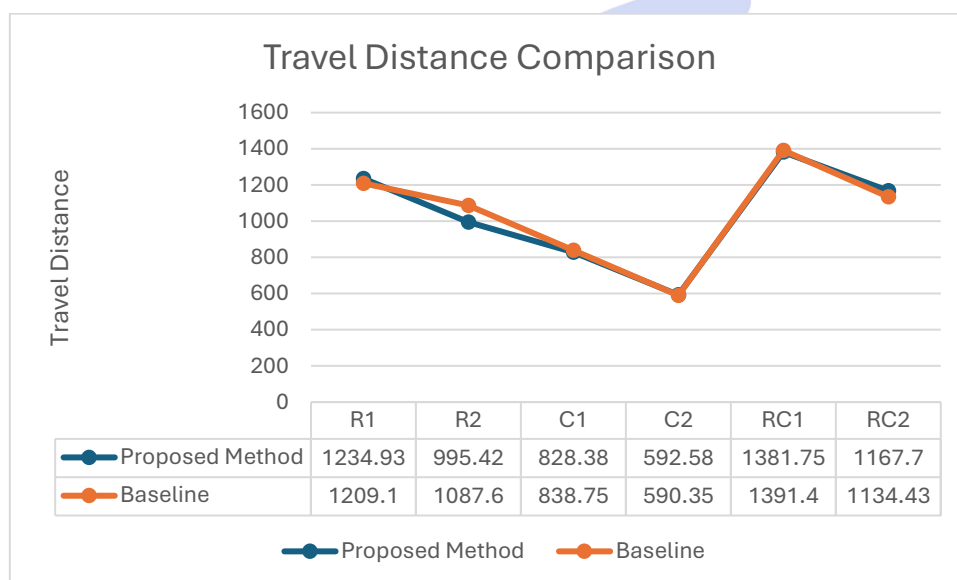


圖 28 本研究與基準方法於各類資料之距離比較

從整體比較結果來看，本研究方法相較於 baseline 於 56 組實例中，有 28 組實例取得較佳結果，同時約 12 組的實例與 Baseline 相同，其餘實例則呈現與基準方法相近之結果。顯示本方法在多數情境下具有相近或更佳解。進一步觀察可發現，本研究方法在多數測試實例中能有效維持或改善車輛使用數，顯示其於路線整合能力方面具有穩定表現，同時在不同問題結構下亦能維持整體解品質。

## 4.7 實驗限制與討論

本研究之實驗結果顯示，所提出之多階段方法於物流公司實務資料上有效改善整體配送結果，在總行駛距離、配送時間及平均裝載率等指標上均較人工規劃結果有所提升。然而本研究所探討問題包含主線店鋪抽取、店鋪分配及支援線規劃三個階段，其中主線店鋪抽取與店鋪分配為基於物流公司實務作業所提出之配送架構。基於此限制，本研究與 Solomon Benchmark Problems 進行比較時，僅針對支援線規劃階段方法進行比較，以評估所提出路線規劃方法之能力。

在實驗設計與求解過程中，由於採用基因演算法與蟻群演算法方法，其結果可能受到隨機性影響，因此在部分接近基準方法之實例中，可能出現輕微波動或表現相似之情形。雖然本研究透過多次實驗或固定隨機種子以降低隨機誤差，但由於搜尋過程中仍包含隨機選擇機制與路線構建順序之差異，實驗結果仍可能呈現些微變異，進而使部分實例之表現略有波動。

此外，本研究之設計主要基於可量化之物流配送條件與限制式進行建構，實務物流營運中仍存在大量難以明確形式化之隱性規則與作業經驗，例如配送人員之臨場判斷、公司內部營運慣例以及特定客戶之服務要求等，並未完整記錄於標準資料結構中，因此難以完全納入數學模型或最佳化演算法中進行處理。本研究雖已涵蓋主要之容量限制、時間窗限制及路線結構限制等核心因素，仍無法保證能完整反映所有物流公司之實務需求與操作細節。

## 第五章 結論與未來研究方向

### 5.1 結論

本研究針對物流配送中固定主線架構下之路線最佳化問題進行探討，提出 Multi-Stage Operationally Constrained VRP 考量實務情境中主線店舖順序固定、需求波動所導致之超載現象，以及需額外啟用支援線進行補充配送之需求，將問題建構為具時間窗限制之多階段組合最佳化問題。

為有效提升求解效率與解品質，本研究將問題拆解為主線超載店舖抽取、候選店舖分配及支援線規劃三個子問題，並依其特性分別設計基因演算法與蟻群最佳化方法進行求解，同時結合修復機制以確保解之可行性。實驗結果顯示，本研究所提出之方法於實務物流資料中能有效降低支援線使用需求並提升整體配送效率。此外，於 Solomon Benchmark Problems 上亦展現良好求解能力，特別在降低車輛使用數量方面具有優勢，顯示本方法兼具實務應用價值與演算法效能。

### 5.2 未來研究方向

儘管本研究已針對固定主線架構下之物流配送問題提出求解方法，未來可進一步延伸至更具多樣性與複雜性之物流環境。在問題延伸方面，可考慮將問題延伸至綠色車輛路徑問題，將碳排放、能源消耗或環境成本納入多目標優化考量，以因應永續物流與減碳政策之發展趨勢，提升模型於環境友善物流決策之應用價值。此外，亦可延伸至異質車輛路線規劃問題，除納入不同車型之載重限制外，行駛成本及營運特性，使模型更貼近實務物流之需求。

另一方面，亦可結合人工智慧相關方法，如深度學習與強化學習等，以改善大規模問題下之決策品質與運算效率，進一步提升模型泛化能力與即時決策能力，促進物流產業朝向更智慧化與低成本化之方向發展。

## 參考文獻

- [1] P. Toth and D. Vigo, *The vehicle routing problem*. SIAM, 2002.
- [2] X.-S. Yang, *Nature-inspired metaheuristic algorithms*. Luniver press, 2010.
- [3] J. H. Holland, "Genetic algorithms," *Scientific American*, vol. 267, no. 1, pp. 66–73, 1992.
- [4] M. Dorigo, M. Birattari, and T. Stutzle, "Ant colony optimization," *IEEE Computational Intelligence Magazine*, vol. 1, no. 4, pp. 28–39, 2007.
- [5] C. Blum, J. Puchinger, G. R. Raidl, and A. Roli, "Hybrid metaheuristics in combinatorial optimization: A survey," *Applied Soft Computing*, vol. 11, no. 6, pp. 4135–4151, 2011.
- [6] T. K. Ralphs, L. Kopman, W. R. Pulleyblank, and L. E. Trotter, "On the capacitated vehicle routing problem," *Mathematical Programming*, vol. 94, no. 2, pp. 343–359, 2003.
- [7] K. C. Tan, L. H. Lee, Q. Zhu, and K. Ou, "Heuristic methods for vehicle routing problem with time windows," *Artificial Intelligence in Engineering*, vol. 15, no. 3, pp. 281–295, 2001.
- [8] J. Euchi, "The vehicle routing problem with private fleet and multiple common carriers: Solution with hybrid metaheuristic algorithm," *Vehicular Communications*, vol. 9, pp. 97–108, 2017.
- [9] P. Chen, H.-k. Huang, and X.-Y. Dong, "Iterated variable neighborhood descent algorithm for the capacitated vehicle routing problem," *Expert Systems with Applications*, vol. 37, no. 2, pp. 1620–1627, 2010.
- [10] A. Duarte, N. Mladenović, J. Sánchez-Oro, and R. Todosijević, "Variable neighborhood descent," in *Handbook of heuristics*: Springer, 2025, pp. 511–539.
- [11] F. Alesiani, G. Ermis, and K. Gkiotsalitis, "Constrained clustering for the capacitated vehicle routing problem (CC-CVRP)," *Applied Artificial Intelligence*, vol. 36, no. 1, p. 1995658, 2022.
- [12] B. Ombuki, B. J. Ross, and F. Hanshar, "Multi-objective genetic algorithms for vehicle routing problem with time windows," *Applied Intelligence*, vol. 24, no. 1, pp. 17–30, 2006.
- [13] A. Maroof, B. Ayvaz, and K. Naeem, "Logistics optimization using hybrid genetic algorithm (hga): A solution to the vehicle routing problem with time windows (vrptw)," *IEEE Access*, vol. 12, pp. 36974–36989, 2024.
- [14] *Solomon's Benchmark Problems*. [Online]. Available: <https://www.sintef.no/projectweb/top/>
- [15] B. Yu, Z. Yang, and B. Yao, "A hybrid algorithm for vehicle routing problem with time windows," *Expert Systems with Applications*, vol. 38, no. 1, pp. 435–441, 2011.

- [16] Y. Zhang and J. Li, "A hybrid heuristic harmony search algorithm for the vehicle routing problem with time windows," *IEEE Access*, vol. 12, pp. 42083–42095, 2024.
- [17] J.-F. Côté and J.-Y. Potvin, "A tabu search heuristic for the vehicle routing problem with private fleet and common carrier," *European Journal of Operational Research*, vol. 198, no. 2, pp. 464–469, 2009.
- [18] M.-C. Bolduc, J. Renaud, F. Boctor, and G. Laporte, "A perturbation metaheuristic for the vehicle routing problem with private fleet and common carriers," *Journal of the Operational Research Society*, vol. 59, no. 6, pp. 776–787, 2008.
- [19] S. Mirjalili, "Genetic algorithm," in *Evolutionary algorithms and neural networks: theory and applications*: Springer, 2018, pp. 43–55.
- [20] S. Katoch, S. S. Chauhan, and V. Kumar, "A review on genetic algorithm: past, present, and future," *Multimedia Tools and Applications*, vol. 80, no. 5, pp. 8091–8126, 2021.
- [21] M. Srinivas and L. M. Patnaik, "Genetic algorithms: A survey," *Computer*, vol. 27, no. 6, pp. 17–26, 2002.
- [22] P. C. Chu and J. E. Beasley, "A genetic algorithm for the multidimensional knapsack problem," *Journal of Heuristics*, vol. 4, no. 1, pp. 63–86, 1998.
- [23] M. Mutingi and C. Mbohwa, "Grouping genetic algorithms," 2017.
- [24] L. Agustí, S. Salcedo-Sanz, S. Jiménez-Fernández, L. Carro-Calvo, J. Del Ser, and J. A. Portilla-Figueras, "A new grouping genetic algorithm for clustering problems," *Expert Systems with Applications*, vol. 39, no. 10, pp. 9695–9703, 2012.
- [25] M. A. Potter and K. A. De Jong, "A cooperative coevolutionary approach to function optimization," in *International conference on parallel problem solving from nature*, 1994: Springer, pp. 249–257.
- [26] X. Ma *et al.*, "A survey on cooperative co-evolutionary algorithms," *IEEE Transactions on Evolutionary Computation*, vol. 23, no. 3, pp. 421–441, 2018.
- [27] A. P. Adedigba, Z. Guo, R. Mallipeddi, and H. Lee, "Cooperative Coevolutionary Genetic Algorithm for Multirobot Task Scheduling in Antarctica region," *Swarm and Evolutionary Computation*, vol. 99, p. 102199, 2025.
- [28] L. M. Gambardella, É. Taillard, and G. Agazzi, "MACS-VRPTW: A multiple ant colony system for vehicle routing problems with time windows," ed: Istituto Dalle Molle Di Studi Sull Intelligenza Artificiale, 1999.
- [29] J. E. Bell and P. R. McMullen, "Ant colony optimization techniques for the vehicle routing problem," *Advanced Engineering Informatics*, vol. 18, no. 1, pp. 41–48, 2004.
- [30] D. Di Caprio, A. Ebrahimnejad, H. Alrezaamiri, and F. J. Santos-Arteaga, "A novel ant colony algorithm for solving shortest path problems with fuzzy arc weights," *Alexandria Engineering Journal*, vol. 61, no. 5, pp. 3403–3415, 2022.
- [31] Q. Ding, X. Hu, L. Sun, and Y. Wang, "An improved ant colony optimization and its

- application to vehicle routing problem with time windows," *Neurocomputing*, vol. 98, pp. 101–107, 2012.
- [32] J. C. Molina, J. L. Salmeron, and I. Eguia, "An ACS-based memetic algorithm for the heterogeneous vehicle routing problem with time windows," *Expert Systems with Applications*, vol. 157, p. 113379, 2020.
- [33] Y. Wang and Z. Han, "Ant colony optimization for traveling salesman problem based on parameters optimization," *Applied Soft Computing*, vol. 107, p. 107439, 2021.
- [34] W. Ma, X. Zhou, H. Zhu, L. Li, and L. Jiao, "A two-stage hybrid ant colony optimization for high-dimensional feature selection," *Pattern Recognition*, vol. 116, p. 107933, 2021.
- [35] N. Frías, F. Johnson, and C. Valle, "Hybrid algorithms for energy minimizing vehicle routing problem: Integrating clusterization and ant colony optimization," *IEEE Access*, vol. 11, pp. 125800–125821, 2023.
- [36] L. Jiao, Z. Peng, L. Xi, M. Guo, S. Ding, and Y. Wei, "A multi-stage heuristic algorithm based on task grouping for vehicle routing problem with energy constraint in disasters," *Expert systems with Applications*, vol. 212, p. 118740, 2023.

